# Carrying Immersion over Networks

A Systematic Review of Networked Extended Reality, 2016–2026

October 2026

**Abstract**

Extended reality (XR) asks more of a network than any earlier medium. A head-mounted display closes a perceptual loop between the user's body and the image: when the head turns, the new view must appear within a few tens of milliseconds. Once content, computation or other participants are elsewhere, the network sits inside that loop. It must carry 360-degree and volumetric media at hundreds of megabits per second, return rendered frames inside a motion-to-photon deadline, take a continuous uplink of pose and sensor data, and keep several people and machines consistent with each other.

This review organises networked XR by the six things every such system must do. It must *encode* the scene into something that can be sent, and *anticipate* what the user will need next. It must decide where to *compute*, and *provision* the network that carries the result. It must *connect* people and machines that share a world, and *assure* that the experience is good enough and safe. For each function we set out the design question, group the approaches into families by their underlying idea, and separate what has been established on real systems from what is contested and what is merely assumed.

Read together, the functions expose a common structure. Each is a trade between network and device: a leaner representation costs decoding, a longer prediction horizon costs accuracy, and moving rendering off the headset costs delay that must be hidden. The functions are also coupled through a shared deadline, so a decision in one sets the budget of the next. Across all six, the same weaknesses recur. Objectives are rarely validated against what users perceive, and requirement figures are inherited rather than measured. Controllers are seldom tested beyond the simulator, and the uplink, multi-user scale and the latency cost of security are studied least. We close with a dependency-ordered research agenda.

## 1 Introduction

A head-mounted display shows a world that must respond to the head that wears it. When the user turns, the new view must appear within a few tens of milliseconds, or the image lags the body and comfort fails [1]. On a standalone device this deadline is met locally. Once the scene, the computation or the other participants are elsewhere, the network becomes part of the perceptual loop.

Extended reality (XR), the umbrella term for virtual, augmented and mixed reality, therefore asks more of a network than video or the web, in four ways. The first is *rate*. A 360-degree or volumetric scene sent whole needs hundreds of megabits per second, although the user sees only a fraction of it at any moment [2, 3]. The second is *deadline*. Remote rendering must return a frame within the motion-to-photon (MTP) budget, so delay is not merely annoying but perceptually disqualifying [1]. The third is *direction*. The device sends pose, camera frames and captured media upstream all the time, so XR is not a download. The fourth is *sharing*. When several people inhabit one world, or one person operates a remote machine, the network must keep endpoints consistent with each other, not just deliver media to each [4].

Research on these demands is scattered across communities that seldom read each other. Multimedia-systems researchers built viewport-adaptive streaming [2, 5]. Mobile-computing researchers offloaded AR recognition to the edge [6]. Wireless researchers framed XR as a joint allocation of radio, computing and energy. Haptics researchers pursued the tactile internet [7], and 6G and metaverse visions placed all of it inside a future network [8, 9]. Existing surveys follow these boundaries. Each covers one medium, such as 360-degree video [2, 5, 10], point clouds [3] or haptics [11], or one technology, such

as edge computing [6, 12], ultra-low latency [13] or the 6G metaverse [8, 14]. What is missing is a single frame that shows how these parts fit together, where they trade against each other, and how far each has been tested.

**Organising idea.** We organise the field by *function*: the six things any networked XR system must do, whatever its medium and access network (Section 2). A system must *encode* the scene into something that can be sent (Section 3). It must *anticipate* what the user will look at or do next, so that the right content arrives in time (Section 4). It must decide where to *compute*, on the device, at the edge or in the cloud, and how to hide the delay this adds (Section 5). It must *provision* the network with links, schedulers, transport and architecture that supply the rate, delay and reliability required (Section 6). It must *connect* users with each other and with remote machines, through shared state, touch and telepresence (Section 7). And it must *assure* that the result is good enough and safe for the people who use it (Section 8). Each function poses its own design question. Within each, we group approaches into a few families by their underlying idea, so that methods described in different vocabularies can be compared on the same terms.

**Contributions.**

(1) A **functional framework** that places the problems of networked XR on one reference architecture and defines each function by a design question (Section 2).

(2) For each function, a **classification of approaches into families**, stating what each family assumes, when it wins and where its limits lie, with the measured results that support each claim (Sections 3 to 8).

(3) An account of the **couplings between functions**: the shared deadline and the trades between network and device that tie them together.

(4) **Nine cross-cutting gaps** and a research agenda ordered by dependency (Sections 9 and 10).

## 2 A Functional Framework for Networked XR

Networked XR is usually described by its parts: latency, caching, 5G, point clouds, holographic communication. These labels mix levels. Latency is a property, caching a mechanism, 5G a technology and holographic communication an application. Lists of such topics therefore do not show how the parts depend on each other. We instead describe the field by *function*, the job a part of the system performs in the loop between the user's body and what the user perceives. Each function answers one design question that any networked XR system must answer, whatever its medium or access network. Figure 1 places the six functions on a reference architecture, and Table 1 summarises the families of approaches that each function contains.

**The six functions.**

**Encode** (Section 3). *What representation of the scene travels, and how is it compressed?* Projections and tiling of 360-degree video, point clouds and meshes, multi-view video, neural scene representations, holograms and haptic signals.

**Anticipate** (Section 4). *What will the user need next, and how is it delivered in time?* Viewport-adaptive streaming, head, gaze and 6DoF motion prediction, and caching and prefetching.

**Compute** (Section 5). *Where does each stage of perception and rendering run, and how is the added delay budgeted and hidden?* Offloading, remote and split rendering, AR inference at the edge, motion-to-photon budgets, reprojection and device energy.

**Provision** (Section 6). *How do radio, scheduling, transport and architecture supply the rate, delay and reliability required?* Resource allocation, mmWave, terahertz, reconfigurable surfaces and Wi-Fi, congestion control and loss resilience, and 6G and metaverse architectures.

**Connect** (Section 7). *How are several people and remote machines kept consistent and responsive?* Multi-user synchronisation, avatars, haptic teleoperation, holographic telepresence and digital twins.

**Assure** (Section 8). *How do we know the result is good enough, and safe?* QoE measurement and modelling, traffic characterisation, and security, privacy and trust.

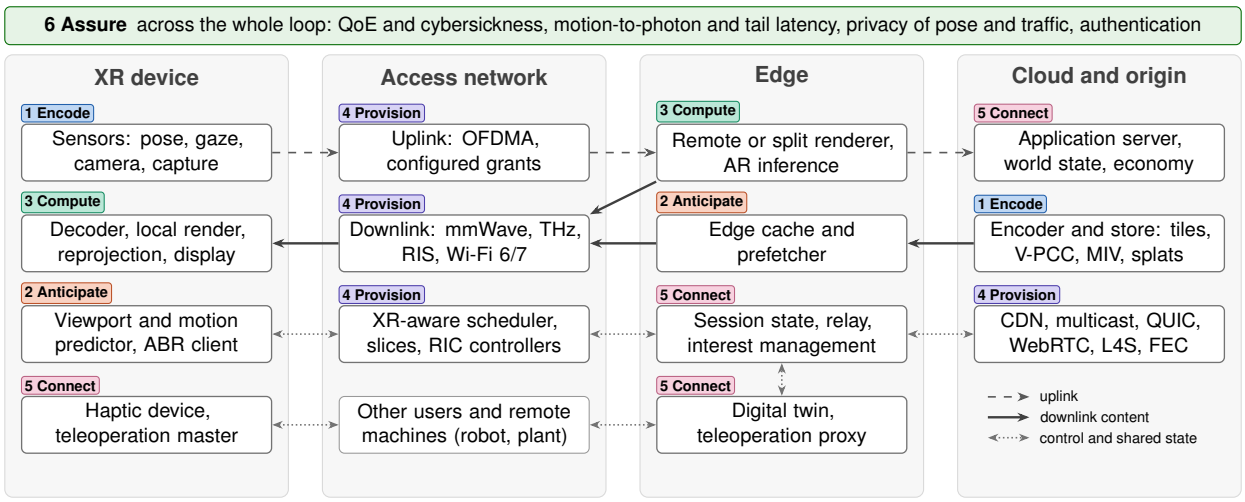



**Figure 1:** Reference architecture of a networked XR system, with the six functions placed where they act. Rows read left to right as the uplink of pose and captured media, right to left as the downlink of content, and as control and shared state that couple users and remote machines. The same function often acts at several tiers: anticipation runs both on the device (prediction) and at the edge (caching), and computation is split between device and edge.

**Why functions.** The grouping has three useful properties. First, each function is *one decision*. Encoding chooses a representation and anticipation a fetch schedule. Computing chooses a split point and provisioning an allocation. Connecting chooses a consistency model and assuring a definition of "good enough". Approaches that answer the same decision can be compared directly, even when they come from different communities. Second, the functions are *ordered along the loop*. Encoding and anticipation decide what is sent, computing and provisioning decide where and how, and connecting and assuring concern the people at the ends. Third, the functions are *coupled*, and the couplings are where most design difficulty lies.

**How the functions couple.** Two couplings run through the whole review. The first is the *shared deadline*. A head-worn display tolerates about 15–20 ms from motion to photon, which leaves roughly 14 ms for computation and communication [1]. Every function spends from this one budget. A longer prediction horizon in anticipation relaxes the network but costs accuracy. A remote split in computing adds a round trip that provisioning must keep short and that reprojection must partly hide. A heavier codec in encoding saves rate but adds encoding delay. The second coupling is the *trade between network and device*. Almost every design moves cost from one to the other. A compact representation must be decoded or synthesised on the headset. Rendering at the edge saves the headset's GPU but spends its radio and battery, and prefetching saves delay but spends bandwidth on content that may never be seen. Reading the sections that follow with these two couplings in mind explains why results rarely transfer between settings. A gain obtained under one budget, or one division of cost, is not a gain under another.

## 3 What Is Sent: Representing and Encoding Immersive Content

Every networked XR system must first decide what its content is: a sphere of pixels, a set of camera views, a cloud of points, the parameters of a learned scene model or a stream of force samples. The design question is which representation travels and how it is compressed. The choice is made before any network algorithm acts, and it fixes four quantities that every later function inherits. The *bitrate floor* is the rate below which quality collapses, and it sets what the network must supply (Section 6). The *encoder delay* is spent before the first bit leaves the sender and counts against the interaction budgets of Sections 5 and 7. The *decoder cost* lands on a battery-powered headset or phone. *Viewport dependence* decides whether the stream can be cut down to what the user will see, and therefore whether prediction and prefetching (Section 4) have anything to act on.

We group representations into four families by what they send (Table 2). *Image-based* representations send what cameras see and reuse the 2D video chain. *Geometry-based* representations send the shape of the scene as points or meshes. *Learned* representations send a model of the scene, or a

**Table 1:** The six functions at a glance. Each row names the design question, the families of approaches compared in the corresponding section and the trade-off that all of them negotiate.

| Function | Design question | Families of approaches | Central trade-off |
|---|---|---|---|
| **Encode** Section 3 | What representation of the scene travels? | Image-based (projected and tiled video, multi-view plus depth); geometry-based (point clouds, meshes); learned (radiance fields, Gaussian splats, semantic codecs); signal-based (haptic, holographic) | Rate on the wire against encoding delay and decoding cost on the device |
| **Anticipate** Section 4 | What will the user need next, and how does it arrive in time? | What to predict (motion, viewport, gaze, saliency); what to fetch (tile and quality selection, heuristic, optimised or learned); where to keep it (buffer, edge cache, multicast) | Prediction horizon against accuracy; bandwidth saved against content wasted |
| **Compute** Section 5 | Where does each stage run, and how is the delay hidden? | Full remote rendering; split and hybrid rendering; perception offload for AR; latency hiding by prediction, reprojection and speculation | Device compute and energy against network delay inside the deadline |
| **Provision** Section 6 | How does the network supply rate, delay and reliability? | Link (mmWave, THz, RIS, Wi-Fi); scheduler (optimisation, heuristics, learning, XR-aware mechanisms); transport (delay-based and L4S control, FEC); architecture (edge, slicing, 6G) | Per-frame guarantees against shared capacity and efficiency |
| **Connect** Section 7 | How are people and machines kept consistent and responsive? | Shared state (authoritative, replicated, interest-managed); people (avatars to holoportation); touch (teleoperation control, haptic coding); places (digital twins, freshness) | Consistency and fidelity against responsiveness and scale |
| **Assure** Section 8 | How do we know it is good enough, and safe? | Experience (user studies, standard, objective and learned QoE models, traffic characterisation); trust (attack surfaces, defences) | Proxy metrics against perceived quality; protection against added delay |

description of its meaning, and let the receiver regenerate the content. *Signal-based* representations send a physical signal, force or light, sampled by the rules of the sense that receives it. Figure 2 follows a chain from each family from capture to display. Adaptation and caching are treated in Section 4, haptic transport and holoportation in Section 7.

### 3.1 Image-based: send what a camera sees

Image-based representations keep the 2D video chain intact and inherit its hardware encoders and decoders. Their questions sit at the two ends of that chain: how the scene is mapped onto rectangles, and how many decoders the client can afford.

*Projected 360-degree video* assumes a viewer who turns the head but stays in place (3DoF), and wins when that holds. The established result is that concentrating bits on the viewport saves bitrate, but less than it saves pixels: Oculus's offset cube used fewer than half the pixels of an equirectangular frame for similar view quality, yet saved only 5.6–16.4% of bitrate [15]. The delivery unit is contested. Viewport-specific versions need no reassembly but multiply storage. Tiles share one encode but cost 7.3–10.4% in coding efficiency [16], up to 37.3% for small tiles at low resolution [22], and the client pays for them: Flare raised phone CPU load from 18% to 58% [16], and streaming took 28.5% of headset power [23], both on 2017–2018 hardware. Gaze is the stronger lever: gaze-based quantisation saved 40–47% in cloud VR gaming in a subjective test [24], and gaze-contingent compression was five times more effective when motion-to-photon (MTP) latency stayed below 15 ms [25]. Foveation is thus a coding gain that Section 5 pays for in latency.

*Multi-view plus depth* sends several colour and depth streams and lets the receiver synthesise the view, which gives limited six degrees of freedom (6DoF) with ordinary video codecs. It assumes the receiver can afford view synthesis and the viewer stays within the span the cameras covered. It wins in live, two-way systems because it needs no slow 3D encoder. Project Starline sends four colour and three depth H.265 streams at 30–100 Mbit/s and fuses them at the receiver, because real-time mesh or point-cloud compression was not possible at its resolution [17]; MetaStream likewise codes RGB-D as 2D video [26]. MPEG Immersive Video (MIV) standardises this path, with atlas packing [27] and view

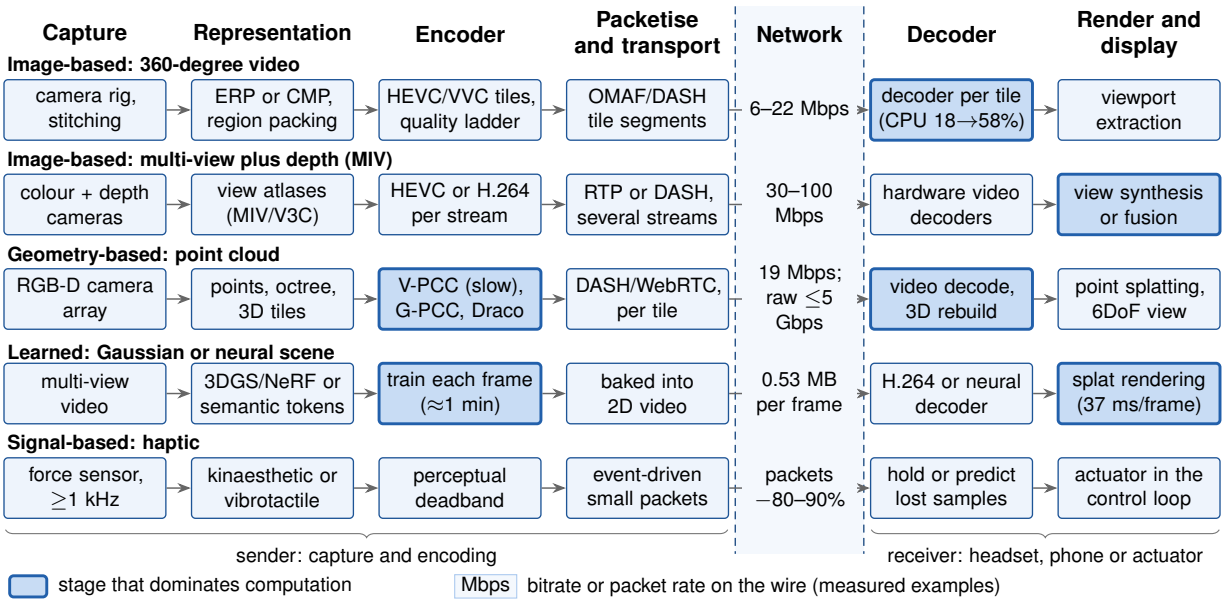



**Figure 2:** Transmission chains from capture to display for one or two representations of each family. No chain is cheap at both ends: a low rate on the wire is bought with encoder delay (V-PCC, per-frame training) or with computation on the receiver (tile decoding, view synthesis, splat rendering). Figures: 360-degree video [15, 16], multi-view streams [17], point clouds [18, 19], Gaussian video [20], deadband coding [21].

synthesis at the decoder [28]. Practice and evidence diverge: deployed telepresence uses this family, yet no study has streamed MIV to users over a real network, and light-field delivery has been tested only offline [29].

### 3.2 Geometry-based: send the shape

Geometry-based representations send the scene itself, so the receiver can render any viewpoint. The price is weight. Raw point clouds need 720 Mbit/s at 200K points and 30 fps [19], and about 5 Gbit/s at one million points [18], one to three orders of magnitude more than 2D video.

Point-cloud codecs trade compression ratio against speed, and the trade is established. V-PCC projects patches onto planes and reuses video codecs; it reaches lossy ratios of 100–500 but took about 4 hours to encode 30 frames of one object [30]. G-PCC and Draco code geometry in 3D, run fast and compress less, so live systems avoid V-PCC. In VR2Gather, the MPEG Anchor codec cut 281.8 Mbit/s to 19.0 Mbit/s but raised latency from 40.6 to 278.7 ms, and parallel tiles brought it back to 90.6 ms [18]: coding, not transit, set the delay. Tuning standard codecs yields single-digit gains (2–6% BD-rate from V-PCC rate control [31]); larger gains come from changing the representation, as when 3D sub-sampling cut bitrate by up to 48.5% at equal quality [32] and 3D patch coding matched V-PCC's ratio with real-time decoding [33]. The receiver is the second constraint: Hermes reached 30 fps on phones only by exploiting inter-frame correlation [34]. No codec wins everywhere; a measurement of Draco, G-PCC, V-PCC and two neural codecs found that the best choice depends on content and viewing behaviour [35].

Geometry's advantage is a richer viewport dependence: points can be culled by visibility, occlusion and distance, not only by direction. ViVo saved 40% of bandwidth on average with SSIM above 0.99, but only 10% for phone users on the same content [19], and gaze-driven delivery on a HoloLens 2 saved up to 67% [36]. Meshes are the less explored member: static mesh codecs have been benchmarked for rate and runtime [37], and a reconstructed mesh for robot teleoperation needed 14 Mbit/s on average [38].

### 3.3 Learned: send a model or a meaning

Learned representations replace explicit content with something the receiver must regenerate. They promise the largest savings and move the most work to the two ends.

*Neural scene representations* (NeRF, 3D Gaussian splatting) send model parameters. The most deployable design reuses existing hardware: $V^3$ bakes dynamic Gaussians into 2D videos for H.264 decoders and reaches 32.97 dB PSNR at 0.532 MB per frame and 27 fps on a phone [20]. That rate

**Table 2:** Representation families and where their cost lands. Figures are examples measured on the cited systems under different conditions; they locate each cost but are not matched operating points.

| Family | Idea | Cost on wire | Cost at sender | Cost at receiver | Maturity |
|---|---|---|---|---|---|
| Image | 360-degree video: sphere projected to rectangles, bits on the viewport | 6–22 Mbps | low; tiles lose 7–10% efficiency | a decoder per tile, CPU 18→58% | standard (OMAF), deployed |
| Image | Multi-view plus depth (MIV): camera views, new views synthesised | 30–100 Mbps | low; hardware video encoders | several decoders and view synthesis | standard; rarely tested on networks |
| Geometry | Point clouds (V-PCC, G-PCC, Draco) | 19 Mbps live; raw up to 5 Gbps | V-PCC hours per second of content; G-PCC, Draco fast | 3D rebuild; 30 fps on phones needs inter-frame coding | standard; on-demand prototypes |
| Geometry | Meshes: surfaces from fused depth | 14 Mbps mean (reconstructed scene) | reconstruction | rasterisation | static codecs benchmarked |
| Learned | Neural scene (NeRF, 3DGS): model parameters per frame | 0.53 MB per frame | training ≈1 min per frame | splat rendering, 37 ms per frame | research prototypes |
| Learned | Semantic and generative: meaning, not pixels | 93–99.9% less | inference, shared knowledge base | generative inference; 13.7 s end to end | simulation, one 5G prototype |
| Signal | Haptic: samples sent only above a perceptual threshold | 80–90% fewer packets | negligible, delay-free | hold or predict missing samples | standard (IEEE 1918.1.1) |
| Signal | Holographic: light field or phase pattern | 6.63 TBps by arithmetic; 173 Mbps measured | V-PCC 11–42 min per second | display not characterised | requirement tables |

is not small: sustained at 30 fps it is about 128 Mbit/s, the range of multi-view video, so the gain is rendering freedom rather than bitrate. Training takes about a minute per frame on a desktop GPU, which rules out live content, and download was measured only on an internal network. Layered Gaussians [39], variable-bitrate NeRF video [40] and splat streaming at 30 fps on phones [41] point towards adaptation; error control and quality assessment remain open [42].

*Semantic and generative codecs* send meaning and assume that sender and receiver share a knowledge base or a generative model. They report the largest savings and the weakest comparisons. WiserVR sent 1.81 Mbit instead of 28.3 Mbit for 20 frames, in an AWGN simulation that excluded the delay of sharing the knowledge base [43]; MagicStream reports up to 1195× less bandwidth [44]. The one generative prototype timed end to end cut bandwidth by 99.93% but raised latency from 980 ms to 13.66 s [45]. Each paper defines fidelity differently, task metrics can disagree with perceived realism [46], and the baseline is usually BPG with LDPC rather than HEVC or VVC. An order-of-magnitude saving is plausible; its price in inference delay and device energy is assumed rather than measured.

### 3.4 Signal-based: send the physical signal

Signal-based representations code a physical quantity by the limits of the sense that receives it. Haptic and holographic content sit at opposite ends of the rate scale.

*Haptic signals* need little rate and strict timing. Kinaesthetic samples travel at 1 kHz or more to keep the control loop stable, and perceptual deadband coding sends a sample only when its change exceeds a Weber fraction (7–15% for force), removing 80–90% of packets without perceptible loss. Two independent implementations of the IEEE 1918.1.1 kinaesthetic codec reproduced the same rates and errors [21], an established result, though for one contact point. Vibrotactile coding is open-loop and may use block transforms with tens of milliseconds of delay [47]. Learned multichannel codecs report cuts of 80–92%, but against each other rather than a standard [48, 49]. Deadband coding also makes traffic bursty, unlike the periodic 1 ms flow that transport studies assume.

*Holographic content* has no agreed representation, and its requirement figures depend on an unstated one. Akyildiz and Guo define a hologram as a recorded light field, then adopt point clouds as the working form; their light-field display example needs 221 GB per frame, or 6.63 TBps at 30 fps [50]. Measured systems used 5–100 Mbps for compressed point clouds over live 5G [51] and 173 Mbps for full-duplex light-field telepresence [52]. The figures do not estimate one quantity: one is uncompressed display arithmetic, the other compressed capture. For live use, codec delay binds first: V-PCC needs 11–42 minutes to encode one second [53]. Optical holograms are only now becoming coded media [54].

**The choice moves cost; it does not remove it.** No representation is cheap at both ends. Image-based content is light on the sender and heavy on the wire and the decoders. Geometry cuts the wire at the price of encoder delay. Learned representations cut it further and move the work into training and inference. Choosing a representation is therefore choosing where to spend: on the network, the sender's encoder or the receiver's processor, and the choice cannot be made alone. Remote rendering (Section 5) collapses any representation back into 2D video at the edge, trading decoder cost for MTP latency. Viewport dependence bounds what prediction can save (Section 4), and encoder delay decides which representations two-way telepresence can use (Section 7). The ratio–speed trade within each family is established; that new representations win across families is assumed, since none has been compared with the others at matched quality.

**Open problems**

**P1. Which representation minimises end-to-end cost for a given scene?** Point clouds, meshes, multi-view video and Gaussian splats have never been compared on the same content at matched perceived quality. Encoding shared scenes in all four and streaming them over replayed 5G and Wi-Fi traces would measure bitrate, encoder delay and decoder load together.

**P2. Do semantic and generative savings survive latency?** Reported savings come from AWGN simulations with weak baselines. Comparing semantic codecs with HEVC, VVC and V-PCC at matched rated quality on a real 5G link, timing encoder and decoder inference, would show whether they suit interactive use.

**P3. How should new representations be judged?** No rating data exist for compressed Gaussian, neural or generative reconstructions viewed in a headset, so PSNR and LPIPS are unvalidated for their artefacts. A 6DoF rating study across codecs and rates would provide ground truth.

**P4. Can codecs track a mobile link?** Rate control is tuned at fixed rates, and neural codecs lack bitrate ladders and loss handling. Rate control driven by measured capacity, judged by frame-deadline misses, would test whether codec gains hold on cellular links.

**P5. What does a hologram actually cost to send?** Requirement tables rest on display arithmetic, not on captured content. Coding one scene as point clouds, splats, light fields and phase holograms and recording the traffic would replace assumed figures with measured ones.

**P6. How do coded haptic streams behave under real loss?** Haptic codecs are evaluated on one-off recordings with one contact point. Shared multi-point recordings and a user study of deadband and vibrotactile codecs over replayed network impairments would test perceptual transparency where it matters.

# 4 Delivering Ahead of the User: Prediction, Adaptation and Caching

An immersive client displays only a fraction of what it could receive, and it must request that fraction before the user looks there. Delivery is therefore a bet on the user's future attention. Three decisions place the bet: *what to predict* about the user, *what to fetch* given that prediction and the network, and *where to keep* content so that it is close when needed. They form one loop (Figure 3). The predictor's horizon limits how deep the buffer may be, the buffer's depth fixes how far ahead the fetcher commits bandwidth, and the caches upstream decide how expensive a wrong commitment is. Table 3 classifies the approaches. Tiling and scene representations belong to Section 3, rendering delay to Section 5, and the radio and transport that carry the fetched bits to Section 6.

## 4.1 What to predict: horizon against accuracy

Prediction trades horizon for accuracy, and the right target depends on what the content lets the user do. For 3DoF video the target is head orientation; for volumetric content it is the full 6DoF pose, because

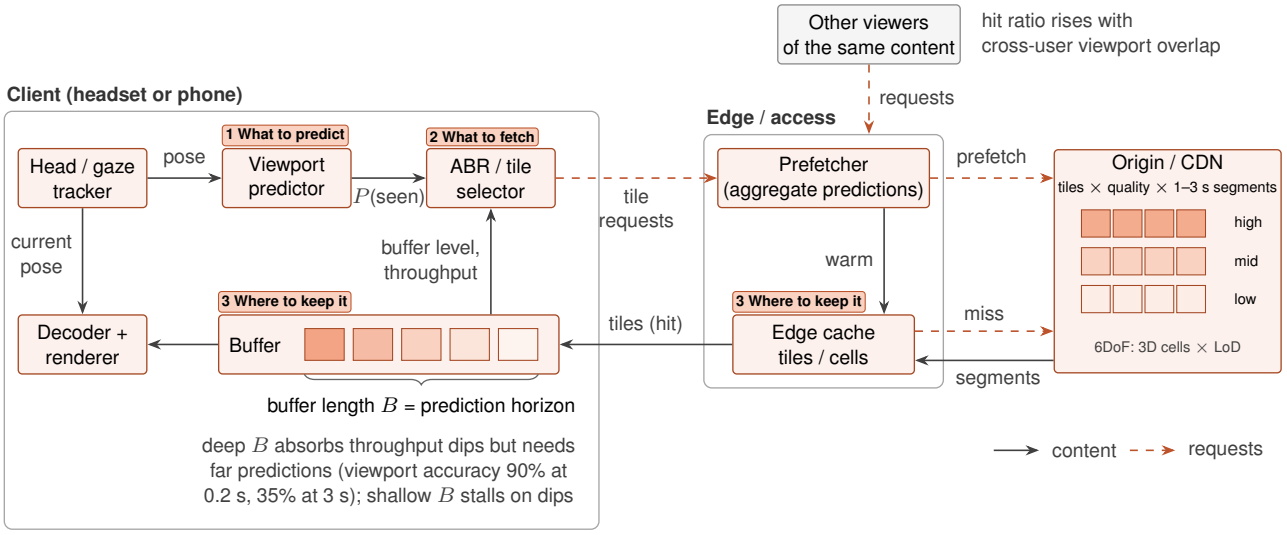



**Figure 3:** Viewport-adaptive delivery of tiled 360-degree or volumetric content across client, edge and origin, with the three anticipation decisions marked. Each decision is a bet on where the user will look: the buffer length fixes how far ahead the predictor must see, and the edge cache pays off only when viewers' viewports overlap.

position, distance and occlusion change what is visible. Gaze narrows the target to the foveal region, while content and other viewers supply priors that do not depend on the user's own history. Accuracy falls steeply with horizon. On a 130-user trace set, Flare's best predictor reached median accuracies of 90.5%, 72.9%, 58.2% and 35.2% at 0.2, 0.5, 1 and 3 s [16], and a cross-dataset benchmark found no algorithm accurate much beyond 300 ms [55], whereas streaming buffers hold seconds of content.

Three families of predictors address this gap. *Extrapolators* (persistence, linear regression, Kalman filters) assume that motion stays smooth over the horizon; they need no training and cost almost nothing, and a Kalman filter predicts 6DoF head pose for cloud-rendered volumetric video [56]. *Learned sequence models* (LSTM, GRU, transformer) learn motion patterns from the user's past trajectory [57]. *Content-aware and cross-user predictors* add what a trajectory cannot know: saliency and motion maps of the frame [58], gaze [59], or the paths of earlier viewers. Earlier viewers are the strongest long-horizon signal for on-demand content. Cross-user prediction raised accuracy by 20.2 percentage points [60], clustering users by fixation added 15% precision over linear regression [61], and other users' paths helped most at 4–10 s [62]. Live content lacks this history and relies on online learning or object semantics [63, 64]; interactive XR lacks both, so these results do not transfer to it.

Whether learned models beat simple ones is contested. On the Tsinghua dataset, persistence (keep the current viewport) reached a hit rate of 0.8355, against 0.8402 for a sequence-to-sequence model and 0.8464 for an LSTM [62]. For 6DoF pose, simple statistical models performed comparably well [65]. Learned models report large gains mostly over other learned models: a fusion transformer reached an average precision of 0.954 at 1 s against 0.901 for the previous best, but on a random split that placed the same videos and users in training and test [66]. Without extrapolation baselines and splits disjoint by video and user, reported accuracy cannot be separated from leakage. A second assumption is that a point estimate is enough. The fetcher needs a distribution to size its margin. Predicting the predictor's own deviation saved 45% of bandwidth at under 0.1% failure [67], and predictors that output several weighted futures exist [68], yet calibration of these probabilities is rarely reported.

## 4.2 What to fetch: turning probabilities into requests

The fetcher turns a prediction and a throughput estimate into requests: which spatial units, at which quality, for which segment. It maximises an objective that is assumed rather than measured, a hand-weighted sum of viewport quality, rebuffering and quality switches. Hedging against prediction error is the common core. 360ProbDASH fetched tiles in proportion to their viewing probability and raised viewport PSNR by about 39% [69], and most designs add low-quality margin tiles around the predicted viewport.

The controllers fall into three families. *Heuristic and buffer-based controllers* map buffer level and view probability to quality with fixed rules; BOLA360 extends buffer-based control to tiles with

**Table 3:** Families of approaches for the three anticipation decisions. Each family wins under its own assumption; none dominates once that assumption fails.

| Decision | Family | Idea | Assumes | Wins when | Limit |
|---|---|---|---|---|---|
| Predict | Extrapolators | Persistence, linear or Kalman continuation of pose | Smooth motion over the horizon | Horizon $<$0.5 s; no training data | Blind to content and to sudden turns |
| | Learned sequence | LSTM or transformer over past trajectory | Training users and videos resemble test ones | Mid horizons on familiar content | Small margin over persistence; leakage-prone splits |
| | Content-aware, cross-user | Saliency, gaze or earlier viewers as priors | Content and viewers are shared | On-demand video at 2–10 s horizons | No prior for live or interactive XR |
| Fetch | Heuristic, buffer-based | Rules map buffer level and view probability to quality | Buffer reflects throughput | Unknown or shifting networks | Hand-tuned; no notion of optimum |
| | Optimisation | Rate-utility allocation over tiles, cells or LoD | Accurate throughput and view probabilities | Good estimates; 6DoF visibility known | Fragile when estimates are wrong |
| | Learned (RL) | Policy learned from trace replay | Deployment matches training traces | Stable, well-sampled conditions | Degrades on unseen users and networks |
| Keep | Popularity cache | LRU or LFU on independent tiles | Requests repeat as whole objects | Large caches; standard HTTP edges | Ignores viewport overlap |
| | Viewport-aware cache, prefetch | Place tiles or objects by predicted viewing | Viewers' attention overlaps | Popular on-demand content | Gains shown only against popularity baselines |
| | Shared delivery | Multicast or coded caching of common parts | Synchronised, similar requests | Co-located viewers of one stream | Asynchrony and subpacketisation |

a near-optimality proof and no training, and reports 6–110% QoE gains [70]. *Optimisation-based controllers* solve a rate-utility allocation per segment, often a knapsack over tiles or 3D cells [71, 72]; they are exact for their model and fragile when its inputs are wrong. *Learned controllers*, mostly deep reinforcement learning, learn the policy from trace replay and report 20–30% gains over heuristics [73, 74]. They win on their training distribution and degrade on unseen viewing patterns and networks [75, 76]. The buffer-based and learned families have never been compared on shared traces under one objective; their ranking is open.

Six degrees of freedom change the unit of adaptation from a tile to a 3D cell, an object or a level of detail (LoD). ViVo combined viewport, distance and occlusion visibility and saved 40% of data on average over commercial 5G at SSIM above 0.99 [19]. No general allocation policy has emerged: the best multi-object strategy depends on the scene and the camera path [77], and the newest public navigation traces come from 15 participants [78].

Live delivery removes both the pre-encoded quality ladder and the cross-user history. YouTube Live delivered 360-degree streams with 4.2–44.2 s camera-to-display delay depending on mode [79]. Two-way volumetric communication breaks the buffer model altogether; such systems reuse low-latency DASH [80] or WebRTC [81]; a tiled user-adaptive point-cloud system needed 14 Mbps instead of about 300 Mbps in a 33-user study [82]. In these systems adaptation is coupled loosely, if at all, to congestion control (Section 6).

### 4.3 Where to keep it: buffers, caches and shared delivery

Keeping content close buys the right to be wrong cheaply. The client buffer holds bets already placed, an edge cache holds what other viewers may need, and shared delivery sends one copy to many. All three gain when viewers' attention overlaps.

*Popularity caches* (LRU, LFU) treat tiles as independent objects, need no prediction and are what

deployed HTTP caches run. *Viewport-aware caches and prefetchers* place tiles by predicted or observed cross-user viewing. Placing tiles by viewport similarity raised the edge hit ratio by at least 40% over LFU and 17% over LRU [83], and OpCASH covered more than 95% of the viewport from cache after 24 views while fetching 85% less data from the origin [84]. Independent groups replicate the advantage [85–87], but always against popularity baselines rather than an oracle such as Belady's. Prefetching extends beyond video: world-scale AR model retrieval missed 20% fewer deadlines with ten times fewer wasted bytes [88]. *Shared delivery* exploits overlap on the air rather than in storage. Multicast of similar viewports saved over 50% [89], and coded caching promises gains that grow with the number of users, although subpacketisation and synchronised requests may erase them [90].

Three questions are open. Fixed tiles, variable tiles that rarely recur [84] and similarity caching over continuous orientations [91] have never been compared. Cache hits distort the throughput estimates of DASH clients [92], yet most caching designs assume an ideal client. Whether learned policies add value cannot be settled while XRFlux remains the only shared benchmark [93]. Beneath all three lies the assumption that hit ratio stands for experience; no XR caching scheme has been evaluated with viewers. Caching of 6DoF content, reusable across viewpoints, users and levels of detail, remains almost unstudied.

### 4.4 One loop: horizon, misprediction and realistic savings

The three decisions share one variable, the time between commitment and display. A deep buffer absorbs throughput dips but commits bits on predictions seconds ahead; a shallow buffer needs only short predictions but stalls when throughput drops. Buffer length is therefore prediction horizon, and at multi-second horizons a deep buffer fills with tiles the user may never see. Short segments of at most 2–3 s ease the tension [94], and upgrading buffered tiles lets the fetcher revise a bet as the prediction sharpens. An edge cache shortens the effective horizon by moving the commitment closer to display, so caching and prediction are complements, not substitutes. The same horizon reappears in remote rendering, where pose prediction hides the network round trip rather than throughput variation (Section 5).

A wrong bet is paid in one of two currencies. A thin margin lets the user turn into low-quality or blank tiles; a wide margin spends the saving. Tiling adds a fixed cost on top: a median coding overhead of 7.3–10.4% for 1-s tiles [16], up to 37.3% for an 8×5 grid [22] (Section 3), and a rise in Flare's phone CPU load from 18% to 58% [16].

The net saving falls as the evaluation becomes real. Perfect knowledge of future motion bounds it at 80% [67]. A scripted viewport gave up to 72% against all tiles at high quality [95]; HEVC tiles saved 65% with a static viewport and over 40% with recorded head motion [22]; prediction with redundancy saved 45% [67]; Flare on phones over commercial LTE, with four users and three videos, saved 26–35% [16]. The gradient recurs in 6DoF: gaze-driven delivery on HoloLens 2 reported up to 67% [36], while ViVo over commercial 5G saved 40% for headset users but only 10% for phone users watching the same content, because people move differently on each device [19]. It is established that viewport-adaptive delivery saves a substantial share of bytes; how large that share is on wireless headsets with real viewers, and whether viewers notice it, is not. Most evidence comes from one on-demand client on an emulated link, only two systems used commercial cellular traces [19, 96], and mainstream quality models failed on a database of 378 viewport-adaptive videos [97]. Several headsets behind one bottleneck, the normal case in a home or a venue, have reached users in no study.

**Open problems**

**P1. Do predictor rankings survive in closed loop?** Persistence nearly matches learned predictors offline, and accuracy is rarely tied to streaming outcome. A tiled player running extrapolators and learned predictors under the same controller, over real bandwidth, with video- and user-disjoint splits and a within-subjects rating study, would show whether accuracy gains reach viewers.

**P2. Can calibrated uncertainty replace fixed margins?** Controllers need view probabilities but receive point estimates. Distribution-free calibration wrapped around any predictor, with margins chosen for a target miss rate, would show how much bandwidth honest uncertainty saves.

**P3. What objective should the fetcher maximise?** Weights of quality, stalls and switches are chosen by hand, and quality models fail on viewport-adaptive streams. Fitting the objective to user ratings, or

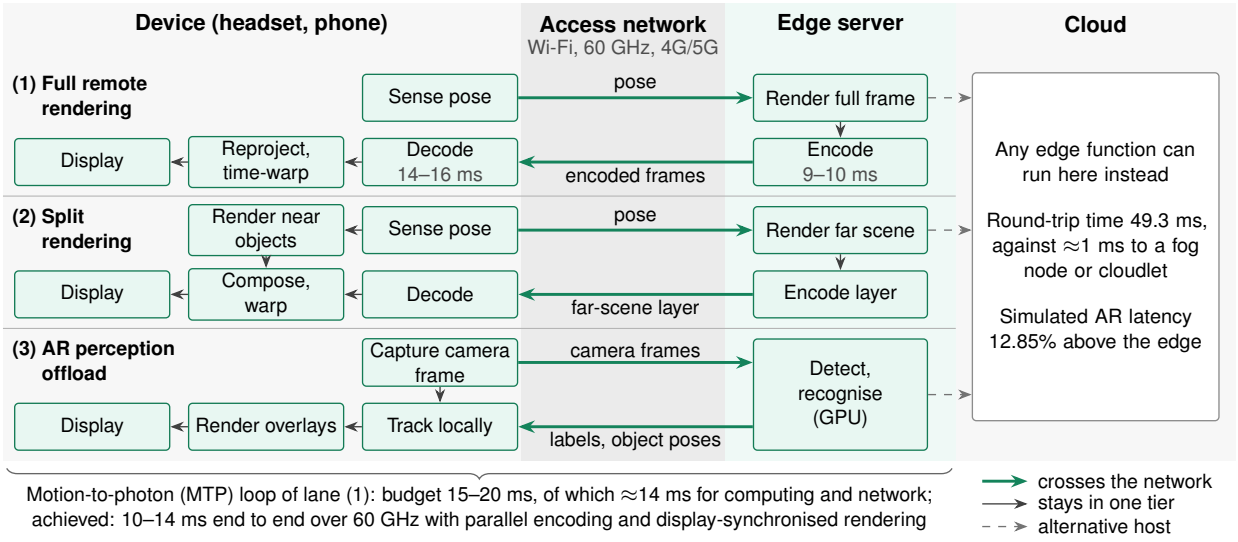



**Figure 4:** Where each pipeline stage runs in full remote rendering, split rendering and AR perception offload. The split point is where the pipeline crosses the access network; display, and in AR also tracking, stay on the device in every design, which is what makes delay hiding possible. Values: MTP budget [1], 60 GHz end-to-end latency [100], encode and phone decode times [101], round-trip times [102], simulated edge advantage [103].

adopting tile extensions of standard models [98, 99], would make controller rankings meaningful.

**P4. How should adaptation behave over real wireless access and shared bottlenecks?** Paired head-motion and radio traces from Wi-Fi and commercial 5G, replayed and run live with several headsets on one access point, would test whether radio hints and access-point coordination reduce stalls and unfairness.

**P5. What should an edge keep for 6DoF content, and do cache gains reach viewers?** A level-of-detail cache shared by co-located users, and an HTTP edge serving tiled DASH under popularity, viewport-aware and Belady policies with viewers, would show whether hit-ratio gains are perceptible.

# 5 Where Computation Happens: Offloading, Remote Rendering and Latency

A headset has a few watts of power and a few milliseconds of slack. Offloading spends the second to save the first: it moves rendering or perception to an edge or cloud server and pays for that move in network delay, which must still fit inside the perceptual deadline of the display loop, the motion-to-photon (MTP) latency. The central design variable is therefore the *split point*, the stage at which the sense–render–display pipeline leaves the device. It fixes what crosses the network, which direction carries the load, how stale the result is when it returns, and which corrections the device can still apply before the photons leave the display (Figure 4). This section organises the field around four decisions: where to split, how to budget the delay a split adds, how to hide that delay, and what the split costs the device. What is sent is treated in Section 3, the prediction that hiding relies on in Section 4, the radio and transport that carry the split in Section 6, and how users perceive delay in Section 8.

## 5.1 Where to split

The later the split, the less the device computes and the more bandwidth and staleness it inherits. Four families of design follow from this tension (Table 4).

*Full remote rendering* moves the whole renderer and streams finished frames. The device keeps only decoding, reprojection and display. Compression is mandatory: raw stereo frames at 2160×1200 and 90 Hz need 5.6 Gbps, beyond any deployed radio [100]. The design is simple and content-agnostic, but every frame is a video frame, so the codec sets a floor on delay and the downlink carries all of the load.

*Split or hybrid rendering* divides the scene. Near or interactive objects are rendered on the device and the distant scene remotely; one such system used 20–30% of the bandwidth of full remote rendering and tolerated delays of up to 200 ms [104–106]. A related group sends an intermediate that the device can re-render from a new pose, not a finished image: colour plus depth for client-side warping

**Table 4:** Families of split point. Each family is defined by what crosses the network; that choice fixes which link carries the load, which delay the device can still hide, and where the design fails.

| Family | What crosses the network | Uplink | Downlink | Hiding available on device | Typical limit |
|---|---|---|---|---|---|
| Full remote rendering | Encoded stereo frames | Pose and input, light | Heavy: compressed video (5.6 Gbps raw) | Rotational and translational reprojection | Codec delay; interaction stays stale |
| Split or hybrid rendering | Far-scene layer, one eye, or a warpable intermediate (depth, atlas, Gaussians) | Pose and input, light | Medium: a fraction of full frames | Local rendering of near objects; 6DoF warping | Device GPU load; disocclusion and composition artefacts |
| Perception offload (AR) | Camera frames up; labels and object poses down | Heavy: video frames | Light: results | Local tracking between detections | Uplink capacity; stale or wrong overlays |
| Placement across tiers | Any of the above, to fog, edge or cloud | As hosted family | As hosted family | As hosted family | Queueing at shared servers; untested learned controllers |

[107], object-space shading atlases [108], one eye rendered remotely and the other reprojected, which lowered MTP by 36% [109], or pixels reused across frames, which removed 95% of frame data [110]. Foveation belongs here too, with a caveat: foveated shading alone did not reduce bitrate, whereas foveated post-processing or encoding did [111]. Neural scenes reopen the choice of intermediate. Current systems stream pixels [112], dynamic Gaussians [113] or partial level-of-detail results [114], each against its own baseline, so the right split for them is not yet known (Section 3).

*Perception offload* is the AR counterpart. The device streams camera frames upward, an edge GPU detects and recognises objects, and only labels and object poses return; tracking and overlay rendering stay local. The load now sits on the uplink. Once the server has a GPU, the network dominates inference latency, and sending frames as H.264 video rather than as images delivered about ten times more frames at 100 kbps [115].

*Placement* decides which server hosts the offloaded stages. Proximity is the one result that holds across methods. In a shipyard deployment, a fog gateway and a cloudlet answered in about 1 ms against 49.3 ms for a remote cloud, but the fog gateway was four times slower than the cloudlet at 240 clients, and the Wi-Fi hop was excluded by design [102]. A simulation found edge execution cut AR latency by 41.44% against local and 12.85% against cloud execution [103]. Static placement breaks down once many users share an edge, because queueing at the server then dominates [116, 117]. Adaptive controllers answer with joint computation–communication optimisation, solved by deep reinforcement learning [118] or Lyapunov control [119]. Their weakness is the inputs they assume. A 2018 fit of detector accuracy against frame size [120] reappears in later formulations [121, 122], VR streams are modelled at a constant 25.32 Mbps [123], and schedulers for several rendering sessions on one GPU exist only in simulation [124, 125]. Learned placement is plausible but almost wholly untested on real render times and real links.

## 5.2 How to budget delay

A latency budget is a sum of stage delays, and it means something only when it names the loop it bounds. Figure 4 shows the stages: sensing, uplink, render, encode, downlink, decode and display. The common decomposition allows under 1 ms for sensing and about 5 ms for the display within a 15–20 ms MTP bound, which leaves about 14 ms for computing and communication [1]. Stage measurements show how tight this is. A phone took 9–10 ms to encode and 14–16 ms to decode an H.264 frame [101], so decoding alone can exceed the network's share.

Pipelining and timing matter as much as link rate. Cutting the Cord split each eye's frame across four parallel encoders and aligned rendering with the client's display refresh. Over 802.11ad it reached 10–14 ms end to end against a sequential median above 26 ms, and missed refreshes fell from 5.3–14.3% to 0.1–0.2% [100]. The uplink is easy to overlook in a frame-centred budget, and in AR it is

the largest stage. In multi-user AR over LTE, median end-to-end latency was 12.5 s, the radio access network contributed 31.2%, and uplink bursts triggered TCP slow starts [126]; on 5G mmWave the uplink was still insufficient for baseline edge detection [127].

Two loops are routinely conflated. MTP runs from head motion to the updated image; interaction latency runs from a user's action to its visible consequence. In a crane simulator, display delays of 0–30 ms affected performance, whereas controller delay mattered only at 800 ms [128]. Quoted targets range from 5 ms [129] and 13 ms [4] to 50 ms [6] and 100 ms [130, 131]. The spread is not noise. It reflects which loop is meant, and whether reprojection or local tracking already absorbs part of the delay.

Budgets are also set on the mean, while users feel the tail. Frame sizes stay bursty even under constant-bitrate coding [132], the 3GPP XR traffic model underestimated effective bitrate by 25% on one headset [133], and on 802.11ac round-trip delay reached 228 ms [134]. A budget met on average can still miss many frames. Direct measurement is cheap, with a photosensor rig [135] or a phone camera with under 10 ms uncertainty [136], yet latency studies seldom report percentiles or jitter. How delay variation translates into experience is taken up in Section 8; the transport and scheduling that shape the tail are in Section 6.

### 5.3 How to hide delay

Every hiding technique lets the last stage on the device correct a result computed for an earlier state. It can therefore correct only what the device observes locally. Four families differ in what that is.

*Reprojection* (time-warp) shifts the last received frame to the newest head pose just before display. It is the established success. Post-render warping removed up to 80% of the aiming penalty of 80 ms added latency [137]; stalls of up to four frames at 90 Hz went unnoticed, and reprojection helped from eight frames upward [138]. With reprojection active, CloudVR users did not notice 20 ms of added network latency [139] and ALVR users hardly noticed round-trip times up to 90 ms [140], although CloudXR quality fell from 75 ms [141]. Reprojection has moved from rotation to translation [142], and selective time-warp leaves head-locked layers untouched [143]. Its limit is fundamental. It corrects the viewpoint, not the scene: moving objects, disoccluded regions and the outcome of a user's action remain stale.

*Prediction and pre-rendering* render for where the user will be. Motion predictors trained on more than 840,000 samples let an edge pre-render frames [144], and rendering beyond the field of view, sized to the current latency, reduced delivery latency by about 22% in a simulation driven by real user data [145]. These methods trade bandwidth for margin, and their gain is bounded by the prediction horizon, which Section 4 treats. *Speculation* goes further and pre-renders every reachable state: Furion prefetched pre-rendered panoramas for each grid point, at about 132 Mbps on average [101]. It works only where the state space is small.

*Local tracking* hides the staleness of remote perception. Without it, 16.7 ms of offload latency raised false detections from 0 to 31.56%; with tracking and region-of-interest encoding, offload latency also fell from 34.56 to 17.23 ms on 2.4 GHz Wi-Fi [146]. CloudAR held 30 frames per second with at least 97% recognition accuracy [147], Jaguar reached about 33 ms end to end [148], and the pattern holds on HoloLens [131]. Replicated across independent groups, this is the design rule of edge AR.

*Hybrid fallback* keeps interactive objects on the device, so the delay-sensitive part never crosses the network [105, 106]. No family hides loss. Users of remote rendering rated packet loss as worse than added delay [140, 149], so a design that hides delay well can still fail on an unreliable link. Nor can any of them hide the state of other users, which Section 7 treats.

### 5.4 What it costs the device

Device energy is the main argument for offloading and the most contested result. Models promise savings of 63% [150] and 85% [122]. Measurements disagree with each other. Edge offloading halved smartphone energy for AR on average [151], and an edge VR testbed lowered client power [152]. A Quest Pro, however, drained its battery 15% faster with wireless offloading than with local rendering [153], and Air Link on a Quest 2 also cost more energy [154]. DeepDecision measured about 2900 mW for offloading regardless of bitrate and frame rate [115], whereas most models scale energy with transmitted data. The likely reason is what the models leave out: the radio's tail energy and the power

of the decoder and display. Offloading moves load from the GPU to the radio and the decoder, and whether that saves energy depends on the device, the codec and the link. None of these studies reports heat, so the thermal side of the trade-off, which also bounds sustained local rendering, is unmeasured.

**Open problems**

**P1. Which split point is best for a given scene, device and link?** Frame, layer, depth and Gaussian splits are each evaluated against their own baseline. A common benchmark with fixed scenes, devices and network traces would rank them and show when each wins.

**P2. Do learned placement controllers survive real render times and real links?** They are trained and tested on synthetic task sizes and channels. Replaying measured render times, head motion, 5G and Wi-Fi traces under the 3GPP XR traffic profile would show which gains hold.

**P3. When does offloading save device energy?** Models predict large savings; measurements on standalone headsets found the opposite [153, 154]. Component-level measurements of radio, decoder, display and GPU across headsets, codecs and links would settle the direction.

**P4. How should a budget account for the tail and for interaction?** Budgets are set on mean MTP, while users feel late frames and delayed consequences of their actions. Studies that vary the delay distribution and the loop affected, with reprojection on and off, would turn targets into derived budgets.

**P5. Can edge AR scale on a shared uplink?** Real-network evidence is single-user and pessimistic [126, 127]. A 5G or Wi-Fi testbed scheduling several AR camera uplinks in one cell would test whether perception offload scales.

# 6 Provisioning the Network: Radio, Resources, Transport and Architecture

XR traffic is unlike the traffic that networks were built for. It is *periodic*: the renderer emits a frame every 11–17 ms and the headset reports pose at least as often. It is *bursty*: each frame leaves the encoder as a block of hundreds of kilobytes that must cross the link before the next one [100], and frame sizes fluctuate even under constant-bitrate encoding [132]. It is *bidirectional*: pose, input and, for edge AR, camera frames go up while rendered content comes down. It is *deadline-bound*: a frame that misses its display slot is worth no more than a lost one. Best-effort, throughput-oriented networks assume elastic flows that tolerate queues and value average rate, and XR breaks each of these assumptions. Provisioning must therefore supply rate, delay and reliability per frame, not per flow, and the tail of frame delivery, not its mean, decides whether the user sees a stall. How much traffic there is depends on where computation is split (Section 5): a remote renderer sends video, a local renderer sends scene state. This section works bottom-up (Figure 5): the link, the scheduler that shares it, the transport that crosses shared bottlenecks, and the architecture that places functions and states what the network must promise.

## 6.1 The link: a directional channel on a moving head

The last hop must carry multi-gigabit bursts to a head that turns and is easily blocked, and its tail, not its peak rate, is the limit. Approaches fall into four families.

*Directional links* (mmWave, THz) buy rate with narrow beams and pay in fragility. THz analysis gives 99.999% reliability at 20 ms with guaranteed line of sight, but with blockage reliability falls to 96% at 20 ms and tail delays reach 30–90 ms while the mean stays below 20 ms [155]. At 60 GHz, 3D head motion defeated beam search designed for 2D [156], and commercial hardware lost up to 4.72% of packets under 45°/s rotation [157]. Blockage and rotation, not physical rate, limit these links.

*Diversity and pose-steered beams* are the remedy that holds across studies. Dual-beam reception with coordinated access points raised reliability by up to 22.8% on real headset motion [158], and distributed antennas kept 12 Gb/s under full rotation [159]. Steering beams from head orientation cuts search overhead [160]; predictive beams from a neural radio field halved beam-search error on a 28 GHz array [161]. These designs reuse the motion predictors of Section 4, so prediction error bounds their gain.

*Reconfigurable surfaces* (RIS) promise to route around blockers, but the promise is assumed. RIS-assisted XR has never run on hardware, learned RIS control violated a 20 ms bound beyond 15 simulated

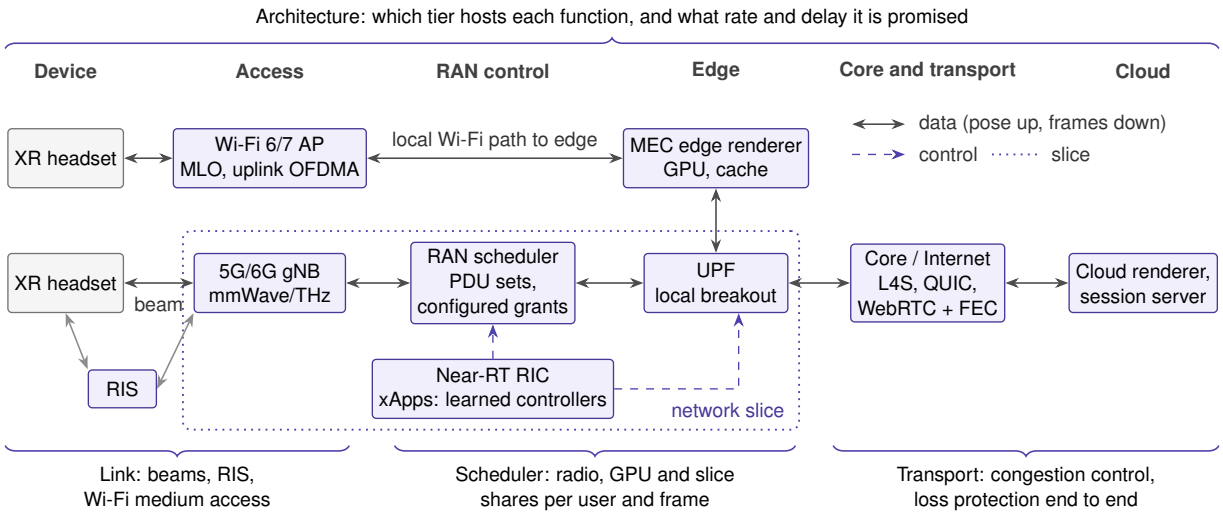



**Figure 5:** End-to-end 5G/6G and Wi-Fi delivery chain for XR, with the layer at which each subsection makes its decisions. The link acts on the last hop, the scheduler inside the slice, transport across the whole path and architecture across all tiers; most designs optimise one layer and hold the others fixed.

**Table 5:** Families of XR schedulers. Each row names what the scheduler decides, what it takes as given, and where it wins and fails.

| Family | Decides | Assumes | Strength | Limit |
|---|---|---|---|---|
| Optimisation (convex, Lyapunov, ADMM) [170, 172, 173] | Cache, render and radio shares per slot | Known cost, QoE and channel models | Bounds; near-optimal at low run time | Breaks under model mismatch; rendering cost assumed |
| Heuristics (greedy, approximation) [174] | Per-user quality level and share | Monotone utility | Simple; provable fraction of optimum; runs on devices | No delay guarantee under bursts |
| Learning (DRL, bandits, xApps) [123, 175–177] | Admission, resource blocks, slice size | Test traffic resembles training traffic | Adapts to unmodelled dynamics | Weak baselines; generalisation and inference cost untested |
| XR-aware mechanisms (PDU sets, XR DRX, CBG-HARQ) [169, 178–180] | Frame-level delay budget, sleep, retransmission | Standard XR traffic model | Frame semantics in the RAN; deployable | System-level simulation only |

users [162], and optimisers that take seconds per slot [163], or minutes on practical prototypes [157], must react within a head turn.

*Shared medium access* binds on Wi-Fi and 5G before physical rate does. With frames aligned to the 802.11ad beacon interval, eight headsets at 400 Mbit/s met a 1 ms bound [164]. Wi-Fi 7 multi-link operation served 7 rather than 6 AR stations at 80 MHz [165], and doubling access point antennas added only 35% capacity [166]. Uplink OFDMA with three headsets disrupted tracking on real hardware [167], and whether uplink or downlink binds is contested because it depends on the assumed rates [165, 168]. On 5G-Advanced, code-block-group retransmission with outer-loop link adaptation reports up to 67% more satisfied XR users [169], in system-level simulation only.

## 6.2 The scheduler: deciding per frame who gets what

A scheduler divides spectrum, power, edge GPU time and cache among users at every slot. These resources substitute for each other: caching rendered fields of view and computing on the device can stand in for bandwidth [170, 171], and joint uplink, downlink and edge allocation saves mobile energy in collaborative AR [150]. The tension is between the model a scheduler optimises and the traffic it meets. Table 5 groups the approaches into four families.

*Optimisation* wins when its models hold: multicasting overlapping 360-degree viewports over mmWave under a Lyapunov delay constraint raised HD delivery by 40–50% and cut 99th-percentile

delay by 17–37% against unicast, in simulation driven by 50 real head traces [172]; ADMM placed objects for edge VR games four orders of magnitude faster than a commercial solver [173]. *Heuristics* trade guarantees for deployability: a collaborative-VR allocator reached at least half the optimum and ran on commodity phones [174]. *Learning* schedulers beat greedy, random and proportional-fair baselines [123, 175], but these baselines are weak, and learning and optimisation are rarely compared on the same traces. On O-RAN testbeds the trade-off becomes concrete: latency-estimation control met 10–16.67 ms targets but over-provisioned, packet-size prediction was efficient but lost latency control [181], and a latency-steering xApp freed 15.8% more resources than an equivalent static slice [177].

*XR-aware mechanisms* change what the radio sees rather than how it optimises. A PDU set groups the packets of one frame so that the RAN can drop or prioritise them together under one delay budget [178]; a delay-tracking PDU-set scheduler served 4–5 times more satisfied users than earlier PDU-set-aware schedulers under scarce resources [179]. Discontinuous reception aligned to the frame period saves headset power [180, 182], and configured grants could pre-allocate the periodic pose uplink. The uplink itself is rarely scheduled for tracking or camera data [123, 183], and no testbed has compared configured with dynamic grants.

Three problems sit above the family choice. *Joint radio–rendering allocation* treats GPU capacity as a free variable [184, 185]; only one testbed adapted base-station bandwidth and edge GPU frequency together [186], and the one measured GPU figure is economic, a 4 GB vGPU slice that halved cloud-XR hourly cost while holding MOS at 4.38 or above [187]. *Multicast* gains depend on viewport overlap [163, 172, 188], so they are unproven for interactive 6DoF content, although one Wi-Fi access point and an edge server served volumetric video to over 30 untethered devices [189]. *The objective* is contested: schedulers optimise a utility of tracking and transmission delay [123], SSIM and VMAF regressions [190], an attention-weighted "Meta-Immersion" score [184] or weighted sums of quality and delay [174]. Gains of 20–37% are reported in each paper's own metric, none validated with users (Section 8). Traffic inputs are optimistic too: the 3GPP baseline model underestimated effective VR bitrate by 25% against a measured headset [133].

### 6.3 Transport: congestion and loss for interactive frames

End to end, frames cross shared bottlenecks where loss-based congestion control fills queues, and a retransmission usually arrives after the frame's deadline. Users find loss the greater threat. In cloud VR, RTT up to 90 ms was hardly noticed while loss up to 8% mattered [140]; with CloudXR, QoE fell at RTT of 75 ms or loss above 6% (N=30) [141]; with Air Link, loss raised cybersickness more than delay did (N=16) [149]. Short stalls of up to four frames at 90 fps went unnoticed, and reprojection helped from eight frames upward [138], which gives transport a margin that the compute pipeline sets (Section 5). Three families address the problem.

*Low-queue congestion control* keeps the bottleneck short. An L4S pipeline for volumetric conferencing converged 45% faster than Google congestion control and lost no frames where WebRTC lost 3% [191], and L4S has been carried over 5G for split rendering [192]. Both used controlled bottlenecks with one competing scheme; radio links with fast capacity swings are untested. *Session protocols* decide how media and control share a connection. Practice is ahead of research: Steam Link carries video and control in one QUIC tunnel [193], Media over QUIC, RTP over QUIC and WebRTC have been compared for holographic conferencing over Wi-Fi and 5G [194], and a WebRTC selective forwarding unit adapted 13–36% faster than TCP for volumetric streams in emulation [195]. On the air, WebRTC batching of frames at fixed intervals wasted Wi-Fi airtime [196], while aligning frame delivery with the display's VSync removed up to 14.3% of frame misses [100]. *Loss protection* repairs damage without waiting for a round trip. Intra refresh reduces latency variability on Wi-Fi [197], but forward error correction for remote rendering has barely been studied [133, 198], although radio losses are bursty and users weigh them most.

### 6.4 Architecture: where functions sit and what the network must promise

Architectures decide which tier hosts rendering, state and caching, and what the network must deliver. Four families recur. *Edge-centric* designs make edge computing a necessary tier, an agreement shared across surveys [8, 14, 199]. *Sliced* designs isolate XR traffic; this family has testbed evidence, since AR, cloud gaming and video calls were measured under RAN-slice configurations on an O-RAN 5G network

**Table 6:** Stated requirements against measured or derived figures. The gaps follow from unstated content representation, codec and control loop.

| Quantity | Stated | Measured or derived |
|---|---|---|
| Per-user metaverse rate | 100–250 Mbps [8, 207]; $\sim 10^3\times$ 5G throughput [199] | Horizon Workrooms 2–3 Mbps down, 0.6 Mbps up, +0.5 Mbps per extra user [205] |
| Holographic call | 1–10 Tbps; 6.63 TBps for a light-field display [50] | 5–100 Mbps for a compressed point cloud over live 5G [51]; <0.2 Mbps for a driven avatar [208] |
| Untethered VR link | 15.75–26.25 Gbps uncompressed [157] | 400 Mbit/s per 4K headset, eight per 802.11ad AP [164] |
| Interaction latency | 1 ms for tactile and telesurgery use [14] | Haptic users tolerated 200 ms with adaptive control [209]; cloud VR RTT up to 90 ms hardly noticed [140] |
| Discomfort threshold | 30 ms [207] vs <20 ms [14] | Not derived from a user study in either source |

[200]. *6G and metaverse visions* add THz access, AI-native control and ledgers [199, 201], but rarely map their functions onto the 5G-Advanced XR mechanisms that already exist [202, 203], so what 6G must add remains unstated. *Measured platforms* tell a plainer story: throughput, latency and device load on five social VR platforms grew almost linearly with users because servers forward avatar data directly [204, 205], and locality-based server assignment scaled VR conferencing to hundreds of users [206]. The scaling bottleneck there is server logic, not access capacity (Section 7).

Requirements need the same discipline as results. Architecture proposals agree on the qualitative list but not on the numbers, and Table 6 shows why: stated per-user rates exceed measured ones by two to five orders of magnitude because the operating point is left unstated. An uncompressed light field is not a compressed point cloud, and 360-degree video is not avatar state. A requirement means something only with its representation, codec, split point and control loop attached. The figures also propagate: one survey reproduces another's requirement table with a reliability target changed from $1-10^{-6}$ to $1-10^{-5}$ [8, 9], so the apparent consensus is partly one source counted several times. One architectural claim remains untested: ledgers are placed in the interactive data path for ownership and trust [8, 201], yet no study has measured the delay they add, although one concedes that replication raises delay with users [210].

**Open problems**

**P1. Which objective should an XR scheduler optimise?** Allocators optimise incompatible QoE constructs that no user study has validated. A QoE-in-the-loop slicing study on an open 5G testbed, comparing static, latency-target and QoE-model xApps with ratings and sickness scores, would decide which objective matters.

**P2. Do joint radio–rendering allocators survive measured costs?** Rendering capacity is a free variable in these models. Profiling render and encode time and GPU power on edge hardware, then re-running published allocators under measured costs, would show whether their rankings hold.

**P3. Can RIS or THz links meet frame deadlines for real XR traffic?** RIS work is simulated and THz reliability rests on Poisson traffic. A pose-driven RIS prototype at 28 or 60 GHz, measured on frame deadline misses under head motion, would test the premise.

**P4. How should the uplink be granted?** Pose and camera uplink enter motion-to-photon latency directly, yet configured, dynamic and pose-prioritised grants have never been compared on a testbed with tracking error as the outcome.

**P5. How should congestion control and loss protection share a frame deadline?** Users weigh loss above delay, but L4S and forward error correction are tested only on emulated bottlenecks. Comparing them over cellular and Wi-Fi with cross traffic, reporting 99th-percentile frame latency, would settle the design.

**P6. What do shared worlds actually require?** Requirement tables are copied, not derived. Traffic measurements of shared-world platforms across user counts and avatar fidelities, compared with standard XR traffic models, would replace them.

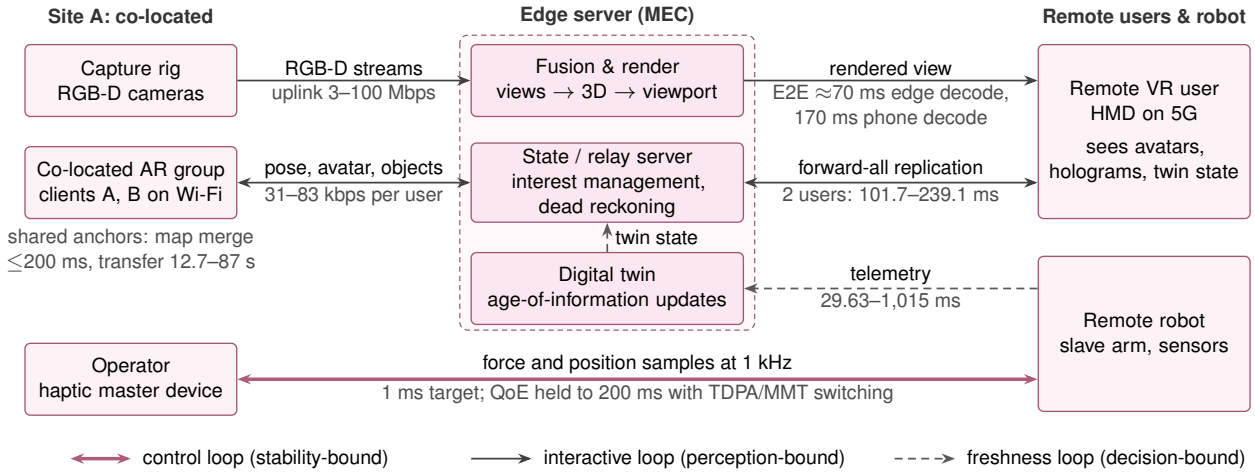



**Figure 6:** Architecture of a multi-party XR session: a co-located group and a remote user share state through an edge relay, a capture rig feeds edge fusion for holoportation, and an operator drives a remote robot whose digital twin updates every viewer. The loops differ in what bounds them (line style), so no single latency target serves the session. Numbers are measured values cited in the text: avatar rates and replication latency [204], holoportation rates and latency [51, 211], anchors [212, 213], twin updates [214, 215], haptic rate and tolerance [21, 209, 216].

# 7 Connecting People and Things: Shared Worlds, Touch and Telepresence

Once several people or machines share one world, delivering media is not enough. The network must keep endpoints consistent and responsive to each other: a gesture made at one site must arrive soon enough to be answered, and an object moved by one user must not rest elsewhere for another. Three kinds of loop run through such a session, and different things bound them (Figure 6 and Table 7). A control loop, such as bilateral haptic teleoperation, closes a feedback system across the network, so delay threatens stability before it threatens experience. Interactive loops carry shared state and remote people; perception bounds them, because delay matters when users notice it or when it changes how they act together. Freshness loops keep a digital twin in step with a physical asset; the decision the viewer must take bounds them. The section is organised by what is shared: state, people, touch, and machines. Haptic and holographic representations belong to Section 3, the radio and transport beneath these flows to Section 6, and group experience to Section 8.

## 7.1 Shared state: who holds the truth, and who sees what

Every shared world must decide where the authoritative copy of its state lives and how much of it each client receives. Three families answer differently. *Authoritative relays* order all updates at a server and forward them. Deployed social VR uses the simplest form: five commercial platforms forward every avatar update unprocessed, so throughput, latency and client load grow roughly linearly with users, and two-user end-to-end latency was 101.7–239.1 ms [204]. A relay gives one ordering of events but puts itself in every path; routing through a cloud middleware raised round-trip time from 0.04 s to 0.6 s [219]. *Replicated, optimistic state* lets each client apply its own actions at once and reconcile later. Croquet-style virtual time [220] and decoupled edge physics serving over 100 users [221] belong here. They buy responsiveness and pay in conflicts: two users actuating one digital twin without a coordination policy conflicted in 33–56% of time windows [222]. *Peer-to-peer and information-centric sharing* removes the relay. Named-data synchronisation [223], device-to-device exchange for web AR [224] and co-located multicast, which rebuilt scenes several times faster than unicast TCP for groups of 7 and 16 [225], take this route. No study has compared the three families on the same workload.

Which inconsistencies users notice is assumed, not measured; consistency models appear only as concepts [226, 227]. Augmented Virtual Teleportation deliberately left its 1.2 s live video unsynchronised with avatar state to stay responsive [228]. Yet synchronisation faults were the most common symptom in 2,649 bug reports from multi-user XR software [229].

Scaling fails at replication before it fails at link capacity. In a controlled study, update intervals grew from 18 ms with 2 clients to 158 ms with 125, and perceived fluidity collapsed beyond 25 avatars [230]. Networked games use two remedies. Interest management sends each client only the state

**Table 7:** The three loop types of a shared XR session. Measured tolerances come from the studies cited in the text; they show that each loop fails in a different way and needs its own network target.

| Loop | What it carries | Bound | Delay measured or tolerated | Failure mode |
|---|---|---|---|---|
| Control (haptic tele-operation) | Force and position samples at 1 kHz | Stability of the bilateral controller | 1 ms target; stable to 200 ms with TDPA/MMT switching [209], model-mediated to 1 s round trip [7] | Oscillation or unsafe contact force |
| Interactive: shared state | Pose, avatar and object updates, tens of kbps per user | Perception of consistency and co-presence | 101.7–239.1 ms on deployed platforms [204]; impairment noticed, task time unchanged [217] | Divergent views, conflicting edits, unfair interaction |
| Interactive: remote people | Avatar parameters to volumetric video, 0.2–100 Mbps | Perception of the person | 70–170 ms end to end [51]; avatar latency noticed at about 320 ms [218] | Degraded, frozen or vanished person |
| Freshness (digital twin) | Telemetry and state of a physical asset | The viewer's decision | 29.63–1,015 ms, all reported adequate [214, 215] | Decisions taken on stale state |

near or relevant to it. Dead reckoning sends a motion model and updates it only when the receiver's extrapolation error exceeds a bound. XR synchronisation has barely adopted either, and neither has been tested with XR users; lossy compression of joint rotations, a partial step, cut delay from over 300 ms to under 25 ms at 100 Hz [231]. Large-scale claims rest on dummy clients [232].

Co-located AR adds a shared coordinate frame, which becomes the bottleneck. SPAR cut spatial inconsistency by up to 60% by choosing what to communicate [233], and edge map merging finished within 200 ms [212], yet anchor transfers took 12.7–87.18 s [213] and multi-user AR over public LTE had a 12.5 s median end-to-end latency [126].

Fairness between users is the most contested question. Edge schedulers minimise inter-player delay [234, 235], and predictive buffering cut inter-client skew by 76% for 100 clients in a controlled testbed [236]. They assume that equal delay means equal experience. User evidence points elsewhere: dyads perceived network impairment but completed tasks no slower [217]. The largest controlled study used 30 pairs [237], and a review of 56 studies found that almost all optimise latency for an individual, not a group [238]. Group quality of experience (QoE), treated in Section 8, is unmeasured beyond pairs.

## 7.2 People: a ladder of fidelity

A remote person can be sent as a few joint angles or as a live volumetric capture; each rung up buys realism with bits, latency and capture hardware. *Pose and parametric avatars* are cheapest: 24 B per update for a head pose and 216 B for a 17-joint skeleton [230], or 31–83 kbps per user on VRChat, Rec Room and Hubs [204]. Expressiveness multiplies the rate; Horizon Worlds exceeded 4.5 Mbps downstream at 15 users [204]. *Semantic, model-driven avatars* send features that drive a pre-built model of the person. Mon3tr animated a Gaussian avatar with under 0.2 Mbps at about 80 ms [208], and MagicStream cut bandwidth up to 1,195-fold by sending keypoint and colour semantics [44]. Apple's spatial personas use about 0.7 Mbps without rate adaptation [239]. This family assumes an enrolled model and a stable feature stream; its robustness to loss is unsettled.

*Volumetric telepresence* captures, uplinks, fuses and renders live geometry, and each stage has its cost. Raw point clouds of one person need about 300 Mbps; tiled user-adaptive streaming brought this to 14 Mbps [82]. An operator trial streamed 5–100 Mbps, with end-to-end latency near 170 ms when phones decoded and 70 ms when the edge did [51]; depth packed into H.264 needed 3–30 Mbps [211]. Edge multipoint control units cut client load as parties grow [240]. At the top, *light-field booths* such as Project Starline [17] need dedicated rigs; a full-duplex light-field link used 173 Mbps on a LAN [52].

Reusing the 2D real-time stack works: a WebRTC selective forwarding unit for point clouds adapted faster than TCP [195], and L4S converged on bandwidth 45% faster than Google congestion control

without frame loss [191]. Two assumptions go unexamined. Calls are treated as downlink problems, yet only two studies measured the holographic uplink [194, 241], and the operator trial attached its camera by Ethernet [51]. Semantic schemes report savings in PSNR, never against geometry at equal bitrate with users. Whether 3D beats 2D is contested. Project Starline [17] and stroke consultations with 30 outpatients [242] favour 3D, while a floating-head prototype [243] and a VR2Gather–Zoom comparison [244] rated 3D below 2D video. All held the network fixed, although humanoid avatars lost more QoE under network distortion than cartoon ones [245].

### 7.3 Touch: closing a control loop across the network

Haptic teleoperation is the only loop here that closes a feedback system, so delay can destabilise it. The field set its requirement early at 1 ms end-to-end latency with 99.999% reliability [216], and IEEE 1918.1 fixed a master–network–slave reference architecture [47]. The requirement is costly at the radio: under finite-blocklength coding, transmit power becomes unbounded once the queueing-delay bound falls below the channel coherence time [246]. Control theory offers three families that relax it. *Wave variables* encode force and velocity so that the channel stays passive, but they handle only constant delay. *Time-domain passivity* (TDPA) monitors the energy the network injects and dissipates it; with perceptual coding it stayed stable under time-varying delay of $100 \pm 30$ ms while sending up to 80% fewer packets. *Model-mediated teleoperation* (MMT) renders force from a local model of the remote environment and sends only model updates; it cut data by about 90% and stayed stable at round-trip delays up to 1 s [7]. MMT assumes an environment that changes slowly enough to be modelled; TDPA suits fast-changing contact at low delay [247].

Perceptual coding is orthogonal to these families. Deadband coders send a sample only when the change would be felt, using Weber fractions of 7–15% for force, and reduce packet rates by 80–90% [21]; codec details belong to Section 3. Prediction carries the same idea to the network: co-designing receiver-side prediction with radio resources saved up to 77.80% bandwidth [248], and octree-based MMT raised force accuracy by 44% with fewer than half the model updates [249]. Both were evaluated open-loop on recorded or synthetic trajectories, not with a live network, a stable controller and users.

Measured tolerance is far looser and depends on the controller. Switching between TDPA and MMT held QoE at delays up to 200 ms [209], and users noticed short haptic delays but compensated [250]. The 1 ms figure assumes a controller without delay compensation; it is not a measured limit of human touch. Transport results split by medium: a time-sensitive networking testbed kept high-priority flows at dedicated-wire quality under 200% over-saturation [251], whereas Wi-Fi needs MAC changes that reached a 3.72 ms worst case for Wi-Fi 7 only in simulation [252]. Haptic streams rarely travel with their video, and cross-modal work questions their usual priority: deliberate haptic loss with visual restoration raised opinion scores by more than 80% at 30% loss [253]. Multi-user haptic studies stop at six pairs [254].

### 7.4 Machines and places: keeping a twin fresh

A digital twin reverses telepresence: the network must keep a replica of a physical asset fresh enough for the viewer's decision. The natural metric is age of information (AoI), the time since the newest received update was generated. Two families use it differently. *Freshness optimisers* allocate radio and edge resources to minimise AoI or synchronisation time. They optimised AoI and reliability for short industrial packets [255], and optimal-transport association cut synchronisation time from 31.23–51.23 ms to 19.11–25.17 ms [256]. They assume their workloads, such as 10 Mb per update [256], and rarely leave simulation; a task-oriented design on a real robot arm, which halved the required packet rate, is an exception [257]. *Measured pipelines* report what deployed stacks deliver. A robot vehicle mirrored on a headset updated in 29.63 ms over a local network [214], but middleware dominated elsewhere: 416 ms per transmission for a robot-arm VR twin [258] and 1,015 ms end to end for a cobot through a commercial IoT platform [215]. Each figure was called adequate; none was set against a task requirement.

Sending state rather than pixels is a consistent lever. Pose streaming kept remote work usable under added latency where video did not [259], a persistent twin with transient object updates allowed teleoperation at kilobits per second [260], and server push used three orders of magnitude less bandwidth than polling for a 5,005-building city twin [261]. Once users act through the twin, the

freshness loop becomes a shared-state problem, with the conflicts seen above [222]. Freshness metrics have multiplied, yet none has been tested against what a viewer perceives or decides.

**Open problems**

**P1. What makes a group experience fair?** Skew schedulers assume equal delay means equal experience, but controlled studies stop at pairs [217, 237]. Groups of two to four under per-user impairments, scored on a shared task and perceived fairness, would give schedulers a validated target.

**P2. Can interest management and dead reckoning scale social XR?** Platforms forward every update and scale linearly [204]. Area-of-interest filtering with error-bounded extrapolation on an open platform, with a perceptual study of avatar error, would show how far replication cost can fall.

**P3. How much delay does touch tolerate in XR tasks?** The 1 ms target persists while users tolerated 200 ms with adaptive control [209]. Factorial delay, loss and jitter studies across TDPA, MMT and closed-loop predictors would yield tolerance curves for provisioning.

**P4. How should haptic flows share a link with XR video?** Haptics and remote-rendered frames are scheduled apart. Joint scheduling on Wi-Fi or 5G, measuring haptic tail delay, motion-to-photon latency and inter-stream skew, would answer it.

**P5. Which rung of the fidelity ladder serves a conversation?** Semantic avatars were never compared with geometry at equal bitrate, and 3D-versus-2D studies hold the network fixed. Two-way calls varying delay, loss and uplink rate against a matched 2D control are needed.

**P6. How stale may a twin be?** AoI is optimised, yet no study links staleness to detection or control errors [215, 256]. Psychometric staleness curves for monitoring and actuation tasks would turn freshness budgets into requirements.

# 8 Assuring Experience and Trust: QoE, Security and Privacy

The previous sections optimise parts of the loop against targets. This section asks whether the result is good enough for its users, and safe for them. Experience and trust share three properties. First, they belong to the whole loop, not to any one component. A frame arrives late because of the encoder, the scheduler, the pacing of the transport and the reprojection on the headset together, and a pose leaks whether it travels to a renderer, a cache or a peer. Second, both are judged at the human end: by what a user perceives, and by what an adversary learns about that user. Third, both are usually inferred from proxies. Network metrics stand in for experience, and attack accuracy on a captured dataset stands in for risk. The section therefore has two halves. On experience, it asks what users actually weigh and how experience and traffic are modelled; on trust, it asks what XR traffic reveals and what defences cost. Figure 7 draws the measurement chain, the attack surfaces and the points where defences add delay.

## 8.1 What users weigh

Users of remote-rendered XR weigh loss and stalls above added delay of tens of milliseconds. Four platforms agree. On ALVR, round-trip times (RTT) up to 90 ms were hardly noticed, while 8% loss and a 35 Mbps cap degraded experience [140]. With CloudXR (N=30), QoE fell at RTT of 75 ms or more, at loss above 6%, or at 25 ms combined with 4% loss [141]. Loss also outweighed delay for presence and emotion on Quest 3 (N=16) [149], and in social XR an RTT up to 77 ms had no significant effect while jitter did [262].

The headset separates two latencies. Rotation latency, the motion-to-photon (MTP) delay of turning the head, is served locally by reprojection; network RTT reaches only interaction latency, the delay of aiming, grabbing or seeing a peer react (Section 5). Isolated stalls of up to four frames went unnoticed, and reprojection helped from eight frames (N=29) [138]. This explains a contested budget. Requirement papers derive 5–10 ms RTT and 20 ms interaction latency from physiology [263], a factor of four to ten below what users tolerated. Reprojection plausibly reconciles the two numbers, but no study has varied RTT and latency compensation together, so the budget that edge placement should meet remains open.

The evidence also separates the outcomes that the network moves from those it does not. Viewport-adaptive delay lowered perceived quality but barely changed presence and left sickness almost untouched (20 observers) [264]. Fewer than 22% of viewers reported sickness for AR point clouds [265],

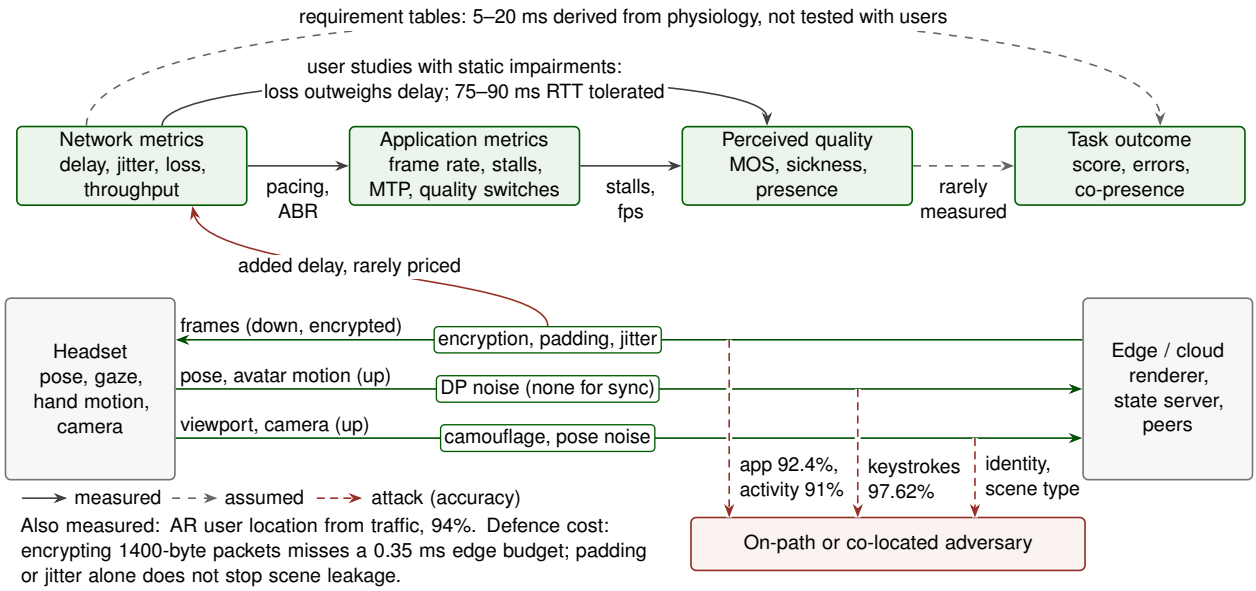



**Figure 7:** Measurement chain and attack surfaces of a networked XR session. Solid arrows are links that user or traffic studies have measured, dashed arrows links that are assumed; red taps show measured attack accuracies, and defences sit on the flows where they add delay that has rarely been priced against the frame deadline.

and it was "not an issue" for 360-degree video [266]; content motion and display, not the network, seem to drive it. For volumetric content, frame rate dominated in a 102-participant study [267], and two quality levels were indistinguishable at 5 m on a HoloLens 2 (N=32) [265]. Task performance, the outcome that matters for training or teleoperation, is rarely measured beside ratings. Group effects are tested almost only in pairs with the impairment on one link [245]. Players blamed their opponent for their own delay [268], and loading avatar and world models congested shared networks as groups grew [269]; who is delayed in a larger group, and how the others judge it, is unknown (Section 7).

These results share a study design: *static impairments*. Every user study applies constant RTT, independent loss or throttling with tools such as NetEm. Real 4G and 5G measurements enter only after the fact, as inputs to models fitted under constant impairments [270]. No study has *replayed measured network dynamics* to users. Wireless loss is bursty and correlated, through aggregation, scheduling and handover, so thresholds learned under independent loss may not transfer.

## 8.2 How experience and traffic are modelled

A QoE model maps what can be observed to what a user would report. Four families differ in what they observe and what they were validated against (Table 8). *Parametric standard models* extend ITU-T P.1203 and P.1204.3 to tiled streaming [98, 99], with P.919 as the test method for short stimuli. *Objective media metrics* are established for coding impairments: PSNR tracks MOS for 360-degree video [271], VMAF correlated best on a public dataset [272], and colour PSNR reached $r = 0.84$ for point clouds, depending on content [265]. Neither family sees interaction latency. *Learned estimators* add it but fit locally. A cloud VR gaming model reached a Pearson correlation of 0.93 and, when used to adapt encoding, raised interaction quality by 1.20 MOS [273]; regressions on the CloudXR data reached a mean absolute error of 0.14 [270]. *Network-side estimators* infer quality from what an operator sees: radio traces, encrypted 360-degree traffic from drive tests [274], inter-packet gaps in a programmable data plane [275], or an LSTM on an O-RAN testbed [276]. They are validated against formulas, not people. PERCEIVE predicted a parametric QoE index within 4%, and its authors state that the index was never checked against ratings [277].

All four families lack *temporal pooling*. P.919 uses 10 s stimuli, while presence needs multi-minute tasks [278], and no model pools sub-second network events into the QoE of a session.

Traffic is characterised in two ways. *Measurements of commercial platforms* show that the application stack, not the link, often sets what users see. WebRTC pacing in 5.56 ms batches set frame delay at 6–21 ms over Wi-Fi 6 with an RTT of 1.5 ms [196]. Under 27 Mbps throttling, adaptive bitrate kept cloud VR frame loss at or below 5.12% where a fixed 40 Mbps stream lost 32.51% [279]: adaptation

**Table 8:** Families of QoE models for networked XR. Each family is limited by what it observes and by the ground truth it was validated against.

| Family | Observes | Validated against | Limit |
|---|---|---|---|
| Parametric standard (P.1203, P.1204.3 tiled) | Bitrate, resolution, stalls, quality switches | Ratings of short streaming stimuli | No interaction latency; 10 s stimuli |
| Objective media metrics (PSNR, VMAF, colour PSNR) | Decoded pixels or points against a reference | Ratings under coding impairments | Blind to timing; content dependent |
| Learned estimators | Network and application metrics, game state | Ratings on a few contents in one lab | Fitted and tested on the same small set |
| Network-side estimators | Encrypted traffic, radio and in-network counters | Parametric indices or activity labels | Never checked against users for interactive XR |
| Requirement tables | Human perception limits | Physiology, not networked user studies | 4–10× stricter than tolerated RTT |

converts a throughput shortfall into image quality rather than lost frames. Horizon Workrooms used 2–3 Mbps downlink and 0.6 Mbps uplink [205], two orders of magnitude below the per-user rates in metaverse visions (Section 6). *Statistical traffic models* capture constant-bitrate video well: frame size follows bitrate over frame rate, and frame intervals follow the frame period with light-tailed or log-logistic marginals [279, 280]. They are open loop, however. The 3GPP baseline underestimated effective VR bitrate by 25% [133], and the open RiftCat model matched mean but not 95th-percentile delay [280]. Recent models add head motion [281] or session states [282], but none couples user motion with the stack's rate adaptation and congestion control. Schedulers evaluated on such models misjudge capacity in the tail.

## 8.3 What XR traffic reveals

XR transmits the body. Pose, gaze, viewport requests and camera frames are control signals for the loop and, at the same time, behavioural and biometric data. Encryption hides their content but not their size, timing or recipients, and four attack surfaces follow.

*Traffic metadata* reveals place and activity, and the result is established across platforms and groups. Traffic patterns of a mobile AR application located users with 94% accuracy [283]. On Meta Quest Pro, off-the-shelf classifiers identified applications with 92.4% accuracy and activities with 91%, from less than 10 minutes of traffic per application [284]. *Motion as a biometric* is the sharpest channel. Avatar motion received by an adversary's own client in multi-user VR yielded 97.62% of typed keystrokes across four applications [285], and Wi-Fi channel state alone yielded 69.75% [286]. State synchronisation, which broadcasts head and hand poses to every peer, thus becomes a leakage channel by design (Section 7). *Viewport requests* identify users because prediction-driven tile requests follow individual head motion (Section 4); viewpoint leakage persists even under federated prediction once prediction errors or QoE are uploaded [287]. *The camera uplink* of offloading AR glasses reveals the type of physical scene through packet sizes and timing [288], so the split chosen in Section 5 is also a privacy decision. Finally, the provider is an observer too: about 70% of Oculus data flows were not properly disclosed [289].

## 8.4 Defences and their price

Every defence spends a resource the loop is short of: delay, bandwidth or prediction accuracy, and the price is rarely measured.

*Encryption* is cheap for small packets and expensive at the tightest budgets. Encrypting 1400-byte packets did not fit a 0.35 ms edge budget, while 256-byte packets did [290]. Selective attribute-based encryption cut encryption and decryption time by up to 50% and 80% and kept volumetric content cacheable [291]. Secured VR streaming under a flood held about 30–32 ms in simulation [292]. *Obfuscation* hides metadata at a cost in bandwidth or quality. Padding or jitter alone did not stop scene leakage, and combined defences conflicted with QoS [288]. Camouflaged tile requests need more resources and degrade prediction [293]. Noise remapping reduced the QoE loss of privacy noise by

55–100%, to 0–3% at maximum privacy [294], and client-side pose noise lowered re-identification on real 6DoF traces [295]; in this line the QoE loss is computed from prediction accuracy, and no user has been asked.

*Authentication* schemes for metaverse avatars and tele-surgery are proven in BAN logic, AVISPA or Real-or-Random models and costed in operation counts [296, 297]. None is measured during an edge or platform hand-off, where re-authentication falls on the interaction path (Section 6). *Federated and edge processing* keeps raw data local but does not by itself protect it: federated learning for VR biometric authentication reached below 10% accuracy [298]. *Detection* rarely sees XR traffic. "Metaverse" intrusion detectors report 96–99% accuracy on IoT and enterprise datasets that contain no XR flows [299]; the exceptions are line-rate detection in Tofino switches at up to 99% accuracy with under 5% of switch resources [300] and the first labelled metaverse attack dataset [301]. *Blockchain* is contested: a 10-user pilot kept 88 FPS at 142 transactions per second [302], while most ledger frameworks report no latency at all. What decides the question is whether a ledger write sits on the interaction path.

The privacy–QoE trade-off is therefore modelled, not perceived. No testbed study reports a defence's latency together with QoE or frame rate. A defence should be reported as a point on a privacy–QoE plane: attacker accuracy against added 99th-percentile frame latency and bandwidth, on a real split-rendering pipeline.

**Open problems**

- **P1. Do network-side estimators predict what XR users report?** Estimators target parametric indices, and none addresses interactive remote rendering. A public benchmark pairing encrypted packet traces, client frame logs and per-minute ratings for cloud VR would let estimators be trained and compared against people.
- **P2. Do thresholds from static impairments hold under real wireless dynamics?** Users have only seen constant RTT and independent loss. A study replaying measured Wi-Fi and 5G traces, with reprojection on and off, would fix the network latency budget for remote-rendered XR.
- **P3. How do sub-second events pool into session QoE, and into group QoE?** No model links bursts of stalls or delay to a multi-minute session, and groups beyond pairs are untested. Scripted burst schedules and asymmetric impairments in groups of four to six would answer both.
- **P4. Can a closed-loop traffic model reproduce the tail?** Models fitted to one stack ignore head motion and rate adaptation, and err at the 95th percentile. A generator coupling pose, adaptation and pacing, validated against several real headsets, would make scheduler studies credible.
- **P5. What does a defence cost against the frame deadline?** Measuring TLS, DTLS and QUIC with padding, bucketing and jitter on split rendering, as attacker accuracy against 99th-percentile frame latency, would give providers a price; the same holds for re-authentication during hand-off.
- **P6. Can state synchronisation share avatars without sharing biometrics?** Poses broadcast to every peer reveal keystrokes, and no synchronisation protocol limits this. A protocol that perturbs or coarsens remote poses, tested for keystroke inference, co-presence and sickness, would close the channel.

# 9 Cross-Cutting Gaps

Read side by side, the open problems of the six functions repeat themselves in different vocabularies. A viewport predictor never tested in a closed loop, an offloading controller that never left the simulator and an allocation scheme whose QoE objective was never shown to a user are three instances of one shortfall. We condense the open problems of Sections 3 to 8 into nine themes (Table 9) in three families: the *evidence* that claims rest on, the *objectives* that designs optimise, and the *blind spots* that most designs leave out.

## 9.1 Evidence

The first family concerns what claims are tested against. *Shared data are missing* (T1). There are no public traffic traces for AR offloading, avatar streams, twin-state updates, holographic calls or haptic flows. No 360-degree dataset records head motion, network conditions and ratings in the same session, and no latency benchmark couples motion, network delay and photodiode-measured MTP. Even simple baselines are often absent. A persistence predictor reached a hit rate of 0.8355 against 0.8402 for a sequence-to-sequence model on the same data [62]. *Evaluation stops at the simulator*

**Table 9:** Nine cross-cutting gaps in networked XR, grouped into three families. Each recurs in at least three of the six functions.

| | Gap | What goes wrong | Functions affected |
|---|---|---|---|
| *Evidence* | | | |
| T1 | No open traces or benchmarks | Each design is tested on its own stack and workload; results cannot be compared or reproduced | all six |
| T2 | Evaluation stops at the simulator | Learned and optimised controllers meet synthetic channels, traffic and motion, rarely a real link | anticipate, compute, provision |
| T3 | Standards unused as baselines | Standard traffic models, codecs and QoE models are seldom the reference configuration, and seldom checked | encode, provision, assure |
| *Objectives* | | | |
| T4 | Metrics not linked to experience | "QoE" objectives use chosen weights; user studies apply static impairments, not real dynamics | all six |
| T5 | Requirements inherited, not derived | Rate and latency targets are copied between surveys without naming content, codec, loop or percentile | encode, provision, connect |
| *Blind spots* | | | |
| T6 | The uplink | Pose, camera and capture streams are modelled as small or ignored; measured uplinks are the bottleneck | compute, provision, connect |
| T7 | Multi-user scale | Systems and studies use one to a few users; group effects and fairness are untested | anticipate, connect, assure |
| T8 | Uncosted security | Attacks are measured; defences are not priced in frame latency or QoE | anticipate, connect, assure |
| T9 | Transport under real impairments | Interactive frames inherit assumptions from buffered streaming; loss resilience is rare | compute, provision, connect |

(T2). Whole technology lines, among them reconfigurable surfaces carrying XR, terahertz XR links and metaverse resource markets, have never met hardware. Where learned controllers were tested on unseen users or content, they degraded [75, 76]. *Standards are not the substrate* (T3). Standard XR traffic models appear almost only in radio simulations. When one was checked against measurement, it underestimated cloud-VR bitrate by 25% [133]. The three gaps share a cause. Traces and testbeds are costly to build, and the venues that publish algorithms rarely credit them.

## 9.2 Objectives

The second family concerns what is optimised. *Network metrics are not linked to experience* (T4). Most objectives called "QoE" are formulas whose weights were chosen rather than fitted. Where users were asked, their answers diverged from the proxies. Packet loss outweighed delay in VR cloud gaming [149]. Users of collaborative VR could not tell forms of latency apart [217]. CloudVR users did not notice 20 ms of added latency [139], while offloading designs optimise to budgets of 5–13 ms. *Requirements are inherited* (T5). Visions state 100–250 Mbps per metaverse user [8, 207], while a measured commercial social-VR platform used 2–3 Mbps downstream [205]. Holographic figures range from 5–100 Mbps for compressed calls [51] to 6.63 TBps [50]. Haptic users tolerated up to 200 ms with adaptive control [209], against the 1 ms figure that motivates much tactile-internet work. A requirement that names neither the representation, the codec, the loop nor the percentile cannot be checked.

## 9.3 Blind spots

The third family concerns what is left out. XR sends a continuous upstream, yet the *uplink* (T6) is studied far less than the downlink, and where it has been measured it is the bottleneck. On public LTE, multi-user AR had a median end-to-end latency of 12.5 s, with the radio access network contributing 31.2% [126]. With three headsets on Wi-Fi 6, uplink OFDMA delayed tracking data [167]. *Scale* (T7) is untested. Measured social platforms scale linearly because they forward all avatar data [204], and controlled QoE studies use pairs of users. *Security is not costed* (T8). Attacks are measured and effective: keystrokes were inferred from avatar motion with 97.62% accuracy [285]. Defences, however, are seldom priced in frame latency. *Transport* (T9) has not followed the move from buffered streaming

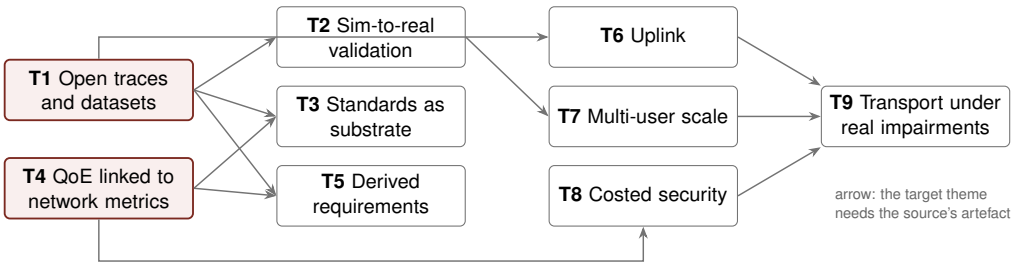



**Figure 8:** Dependencies among the nine themes. Open traces (T1) and a validated mapping from network metrics to experience (T4) are the roots. Without them, simulations cannot be validated, requirements cannot be derived, and allocation, transport and security results cannot be compared in units that matter to users.

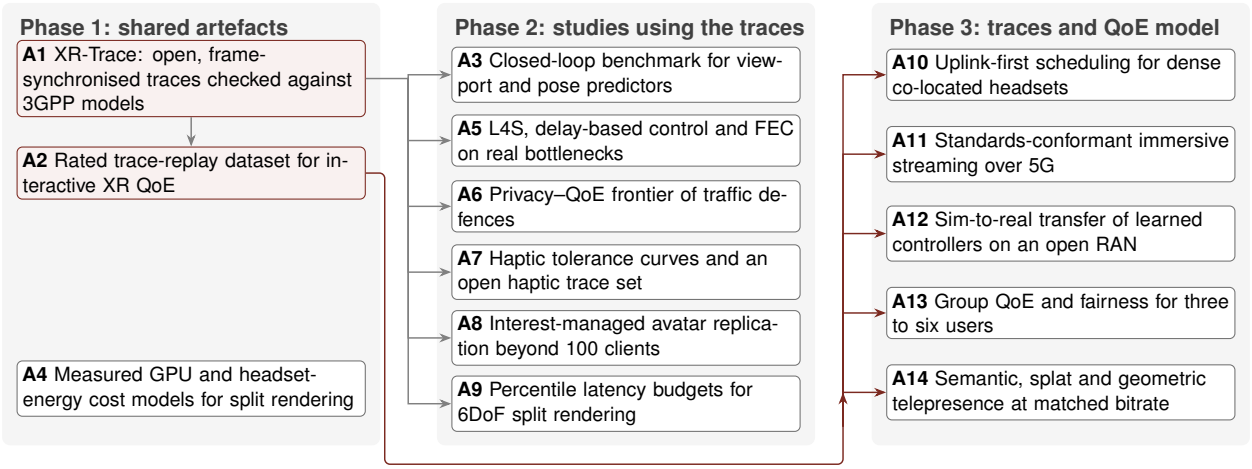



**Figure 9:** Research agenda as a dependency roadmap. Phase-1 projects (red) build the artefacts that the others consume. A1 supplies traces to every later project (grey arrows); A2, built on A1, adds the validated QoE model that the Phase-3 projects optimise against (red arrows). A4 stands alone.

to interactive delivery. Users rank loss above delay [140, 149], yet forward error correction and L4S are rarely applied. An L4S pipeline for volumetric conferencing converged 45% faster than Google Congestion Control [191], but only in a controlled bottleneck.

### 9.4 How the gaps depend on each other

The gaps are not independent (Figure 8). Two are roots. Without open traces (T1), simulations cannot be validated (T2), requirements cannot be derived (T5), standard traffic models cannot be checked (T3), and uplink and multi-user workloads cannot even be described (T6, T7). Without a validated mapping from network metrics to experience (T4), results on allocation, transport and security (T8, T9) cannot be compared in units that matter to users. Two smaller questions point the same way. Device-energy savings from offloading rest on models, and the few measurements disagree: one standalone headset drained its battery 15% faster when offloading wirelessly [153], while an edge VR testbed found lower client power [152]. Neural scene representations have no tested rate adaptation, cache or split point. Both questions need the same two roots, and this ordering shapes the agenda that follows.

## 10 Research Agenda

The gaps of Section 9 suggest fourteen projects that a networking or multimedia laboratory could start with existing hardware and open software. We order them by dependency (Figure 9). Projects that produce shared artefacts come first, because the others need them.

**Phase 1: build what others need.** *XR-Trace* (A1) would instrument the main workloads on 5G standalone and Wi-Fi 6E/7, with one, four and eight users. These are split-rendered VR, AR camera offloading, avatar and shared-state traffic, tiled 360-degree video and twin-state updates. Each trace would record pose, encoder output, packets at both ends, application events and photodiode MTP on a common clock. The project would then fit traffic models, compare them with the 3GPP XR models, and release the traces with a replay harness for ns-3 and NetEm. It comes first because every later project consumes it. The *rated replay dataset* (A2) would have participants perform interactive tasks while

the network replays A1's traces. It would collect momentary and session ratings, sickness scores, task performance and packet logs together, so that a QoE model can be fitted and then used as the objective by other work. It would also measure the difference between replayed dynamics and independent impairments of the same mean. *Energy cost models* (A4) would replace the assumed GPU and battery costs inside offloading optimisers with measured ones.

**Phase 2 and 3: answer the open questions on shared ground.** With traces in hand, five studies become cheap and comparable. They are a closed-loop predictor benchmark with persistence and Kalman baselines (A3), transport on real bottlenecks (A5), a privacy–QoE frontier for padding and jitter defences (A6), haptic tolerance curves (A7) and interest-managed replication beyond one hundred clients (A8). A percentile latency budget for 6DoF split rendering (A9) completes the phase. With a validated QoE model as well, the remaining projects can optimise against experience rather than a proxy. These are uplink-first scheduling (A10), a standards-conformant 5G pipeline (A11), sim-to-real transfer of learned controllers as xApps on an open RAN (A12), group QoE and fairness (A13), and a matched-bitrate comparison of semantic, Gaussian-splat and geometric telepresence (A14).

**The evaluation bar.** Each project should meet the standard that Transactions-level reviewers apply, and that much current work does not yet meet. First, a *baseline ladder*: a trivial baseline, a classical method, the strongest recent method re-implemented under the same conditions, an ablation and, where computable, an oracle. Second, *realism with control*: real hardware for the claim and trace replay for repeatability, with the gap between simulation and measurement reported. Third, *out-of-distribution tests* for anything learned, on held-out users, content and networks. Fourth, *sound statistics*: repeated runs with confidence intervals and, for user studies, a power analysis, within-subjects designs and mixed-effects models. Fifth, *reported cost and released artefacts*: run time, energy and signalling overhead on the target device, with code, traces and analysis scripts published.

# 11 Conclusion

Networked XR puts the network inside a perceptual loop. That single fact explains why its problems differ from those of video streaming or cloud gaming. Content must be chosen before the user looks at it. Rendering can move off the headset only if its delay can be hidden. Radio and transport must deliver frames, not flows, on time. Several people and machines must agree on one world. And quality must be judged at the human end, where both experience and privacy are decided.

Organising the field by function makes its structure visible. Six design questions cover it: what to encode, what to anticipate, where to compute, how to provision, how to connect and how to assure. Each has a small number of families of answers. Two couplings tie the functions together. The first is a shared deadline that every function spends from. The second is a trade between network and device that almost every design shifts one way or the other. Results rarely transfer between settings because they were obtained under a particular budget and a particular division of cost.

The weakest links are not algorithms but shared ground. The field lacks open, frame-synchronised traces of real XR workloads, and a model of experience fitted to people under realistic network dynamics. With these two artefacts, the questions that recur in every section become answerable on common terms: whether a learned scheduler beats a heuristic on a real cell, how much latency a defence costs, and what a hologram really needs. Without them, networked XR will keep producing designs faster than it can tell which of them work.

# References


[1] Mohammed S. Elbamby, Cristina Perfecto, Mehdi Bennis, et al. Toward Low-Latency and Ultra-Reliable Virtual Reality. *IEEE Network*, 2018.

[2] Abid Yaqoob, Ting Bi, and Gabriel-Miro Muntean. A Survey on Adaptive 360deg Video Streaming: Solutions, Challenges and Opportunities. *IEEE Communications Surveys & Tutorials*, 2020.

[3] Zhi Liu, Qiyue Li, Xianfu Chen, et al. Point Cloud Video Streaming: Challenges and Solutions. *IEEE Network*, 2021.

[4] Ejder Bastug, Mehdi Bennis, Muriel Medard, et al. Toward Interconnected Virtual Reality: Opportunities, Challenges, and Enablers. *IEEE Communications Magazine*, 2017.

[5] Ching-Ling Fan, Wen-Chih Lo, Yu-Tung Pai, et al. A Survey on 360deg Video Streaming. *ACM Computing Surveys*, 2019.

[6] Yushan Siriwardhana, Pawani Porambage, Madhusanka Liyanage, et al. A Survey on Mobile Augmented Reality With 5G Mobile Edge Computing: Architectures, Applications, and Technical Aspects. *IEEE Communications Surveys & Tutorials*, 2021.

[7] Konstantinos Antonakoglou, Xiao Wen Xu, Eckehard G. Steinbach, et al. Toward Haptic Communications Over the 5G Tactile Internet. *IEEE Communications Surveys & Tutorials*, 2018.

[8] Minrui Xu, Wei Chong Ng, Wei Yang Bryan Lim, et al. A Full Dive Into Realizing the Edge-Enabled Metaverse: Visions, Enabling Technologies, and Challenges. *IEEE Communications*

*Surveys & Tutorials*, 2022.

[9] Mansoor Ali, Faisal Naeem, Georges Kaddoum, et al. Metaverse Communications, Networking, Security, and Applications: Research Issues, State-of-the-Art, and Future Directions. *IEEE Communications Surveys & Tutorials*, 2023.

[10] Michael Zink, Ramesh K. Sitaraman, and Klara Nahrstedt. Scalable 360deg Video Stream Delivery: Challenges, Solutions, and Opportunities. *Proceedings of the IEEE*, 2019.

[11] Nattakorn Promwongsa, Amin Ebrahimzadeh, Diala Naboulsi, et al. A Comprehensive Survey of the Tactile Internet: State-of-the-Art and Research Directions. *IEEE Communications Surveys & Tutorials*, 2020.

[12] Sukhmani Sukhmani, Mohammad Sadeghi, Melike Erol-Kantarci, et al. Edge Caching and Computing in 5G for Mobile AR/VR and Tactile Internet. *IEEE Multimedia*, 2018.

[13] Ananya Hazarika and Mehdi Rahmati. Towards an Evolved Immersive Experience: Exploring 5G- and Beyond-Enabled Ultra-Low-Latency Communications for Augmented and Virtual Reality. *Sensors*, 2023.

[14] Xuemin Shen, Jie Gao, Mushu Li, et al. Toward immersive communications in 6G. *Frontiers in Computer Science*, 2023.

[15] Chao Zhou, Zhenhua Li, and Yao Liu. A Measurement Study of Oculus 360 Degree Video Streaming. In *nan*, 2017.

[16] Feng Qian, Bo Han, Qingyang Xiao, et al. Flare. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, 2018.

[17] Jason Lawrence, Danb Goldman, Supreeth Achar, et al. Project starline. *ACM Transactions on Graphics*, 2021.

[18] Irene Viola, Jack Jansen, Shishir Subramanyam, et al. VR2Gather: A Collaborative, Social Virtual Reality System for Adaptive, Multiparty Real-Time Communication. *IEEE Multimedia*, 2023.

[19] Bo Han, Yu Liu, and Feng Qian. ViVo. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, 2020.

[20] Penghao Wang, Zhirui Zhang, Liao Wang, et al. V^3: Viewing Volumetric Videos on Mobiles via Streamable 2D Dynamic Gaussians. *ACM Transactions on Graphics*, 2024.

[21] Eckehard G. Steinbach, Matti Strese, Mohamad A. Eid, et al. Haptic Codecs for the Tactile Internet. *Proceedings of the IEEE*, 2018.

[22] Mario Graf, Christian Timmerer, and Christopher Mueller. Towards Bandwidth Efficient Adaptive Streaming of Omnidirectional Video over HTTP. In *nan*, 2017.

[23] Nan Jiang, Viswanathan Swaminathan, and Sheng Wei. Power Evaluation of 360 VR Video Streaming on Head Mounted Display Devices. In *nan*, 2017.

[24] Wenjie Zou, Shixuan Feng, Xionghui Mao, et al. Enhancing Quality Of Experience For Cloud Virtual Reality Gaming: An Object-Aware Video Encoding. In *IEEE International Conference on Multimedia & Expo Workshops (ICMEW)*, 2021.

[25] Luke H.C. Hsiao, Brooke Krajancich, Philip A. Levis, et al. Towards retina-quality VR video streaming. *ACM SIGCOMM Computer Communication Review*, 2022.

[26] Yongjie Guan, Xueyu Hou, Nan Wu, et al. MetaStream: Live Volumetric Content Capture, Creation, Delivery, and Rendering in Real Time. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, 2023.

[27] Soonbin Lee, Jong-Beom Jeong, and Eun-Seok Ryu. Efficient Group-Based Packing Strategy for 6DoF Immersive Video Streaming. In *International Conference on Information Networking*, 2022.

[28] Julien Fleureau, Bertrand Chupeau, Franck Thudor, et al. An Immersive Video Experience with Real-Time View Synthesis Leveraging the Upcoming MIV Distribution Standard. In *IEEE International Conference on Multimedia & Expo Workshops (ICMEW)*, 2020.

[29] Maarten Wijnants, Hendrik Lievens, Nick Michiels, et al. Standards-compliant HTTP adaptive streaming of static light fields. In *ACM Symposium on Virtual Reality Software and Technology (VRST)*, 2018.

[30] Jeroen van der Hooft, Maria Torres Vega, Tim Wauters, et al. From Capturing to Rendering: Volumetric Media Delivery with Six Degrees of Freedom. *IEEE Communications Magazine*, 2020.

[31] Zhanyuan Cai, Wenxu Gao, Ge Li, et al. Distortion Propagation Model-Based V-PCC Rate Control for 3D Point Cloud Broadcasting. *IEEE Transactions on Broadcasting*, 2024.

[32] Yuang Shi, Pranav Venkatram, Yifan Ding, et al. Enabling Low Bit-Rate MPEG V-PCC-encoded Volumetric Video Streaming with 3D Sub-sampling. In *ACM Multimedia Systems Conference (MMSys)*, 2023.

[33] R. F. Chen, Mengbai Xiao, Dongxiao Yu, et al. patchVVC: A Real-time Compression Framework for Streaming Volumetric Videos. In *ACM Multimedia Systems Conference (MMSys)*, 2023.

[34] Yizong Wang, Dong Yang Zhao, Huanhuan Zhang, et al. Hermes: Leveraging Implicit Inter-Frame Correlation for Bandwidth-Efficient Mobile Volumetric Video Streaming. In *ACM International Conference on Multimedia (ACM MM)*, 2023.

[35] Mengyu Yang, Zhenxiao Luo, Miao Hu, et al. A Comparative Measurement Study of Point Cloud-Based Volumetric Video Codecs. *IEEE Transactions on Broadcasting*, 2023.

[36] Nan Wu, Kaiyan Liu, Ruizhi Cheng, et al. Theia: Gaze-driven and Perception-aware Volumetric Content Delivery for Mixed Reality Headsets. In *International Conference on Mobile Systems, Applications, and Services*, 2024.

[37] Alexandros Doumanoglou, Petros Drakoulis, Nikolaos Zioulis, et al. Benchmarking Open-Source Static 3D Mesh Codecs for Immersive Media Interactive Live Streaming. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2019.

[38] Patrick Stotko, Stefan Krumpen, Max Schwarz, et al. A VR System for Immersive Teleoperation and Live Exploration with a Mobile Robot. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.

[39] Yuang Shi, Geraldine Morin, Simone Gasparini, et al. LapisGS: Layered Progressive 3D Gaussian Splatting for Adaptive Streaming. In *International Conference on 3D Vision (3DV)*, 2025.

[40] Zihan Zheng, Houqiang Zhong, Qiang Hu, et al. HPC: Hierarchical Progressive Coding Framework for Volumetric Video. In *ACM International Conference on Multimedia (ACM MM)*, 2024.

[41] Gunjoong Kim, Seonghoon Park, Jeho Lee, et al. Vega: Fully Immersive Mobile Volumetric Video Streaming with 3D Gaussian Splatting. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, 2025.

[42] Yuang Shi. 3D Gaussian-based Immersive Media Streaming in Networked Extended Reality. In *ACM Multimedia Systems Conference (MMSys)*, 2025.

[43] Le Xia, Yao Sun, Chengsi Liang, et al. WiserVR: Semantic Communication Enabled Wireless Virtual Reality Delivery. *IEEE Wireless Communications*, 2023.

[44] Ruizhi Cheng, Nan Wu, Vu Le, et al. MagicStream: Bandwidth-conserving Immersive Telepresence via Semantic Communication. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2024.

[45] Nassim Sehad, Lina Bariah, Wassim Hamidouche, et al. Generative AI for Immersive Communication: The Next Frontier in Internet-of-Senses Through 6G. *IEEE Communications Magazine*, 2024.

[46] Bowen Zhang, Zhijin Qin, and Geoffrey Ye Li. Semantic Communications With Variable-Length Coding for Extended Reality. *IEEE Journal of Selected Topics in Signal Processing*, 2023.

[47] Oliver Holland, Eckehard G. Steinbach, Ramjee Prasad, et al. The IEEE 1918.1 "Tactile Internet" Standards Working Group and its Standards. *Proceedings of the IEEE*, 2019.

[48] Jie Song, Yun Gao, Dan Wu, et al. Scalable Tactile Coding. *IEEE Transactions on Multimedia*, 2026.

[49] Tom Ogura, Shogo Kitamura, Takuya Fujihashi, et al. Joint Source-Channel Coding for Multi-Channel Vibrotactile Sensors. *IEEE Access*, 2025.

[50] Ian Fuat Akyildiz and Hongzhi Guo. Holographic-type communication: A new challenge for the next decade. *ITU Journal on Future and Evolving Technologies*, 2022.

[51] Ali El Essaili, Sara Thorson, Alvin Jude, et al. Holographic communication in 5G networks. *Ericsson Technology Review*, 2022.

[52] Christopher Blackwell, Javid Khan, and Xianzhong Chen. 54-

6: Holographic 3D Telepresence System with Light Field 3D Displays and Depth Cameras over a LAN. In *SID Symposium Digest of Technical Papers*, 2021.

[53] Yakun Huang, Yuanwei Zhu, Xiuquan Qiao, et al. Toward Holographic Video Communications: A Promising AI-Driven Solution. *IEEE Communications Magazine*, 2022.

[54] Shuyang Xie, Jie Na Zhou, Jun Wang, et al. JPEG-Inspired Cloud-Edge Holography. *arXiv (Cornell University)*, 2025.

[55] Shahryar Afzal, Jiasi Chen, and Kadangode K. Ramakrishnan. Viewing the 360deg Future: Trade-Off Between User Field-of-View Prediction, Network Bandwidth, and Delay. In *Proceedings/Proceedings - International Conference on Computer Communications and Networks*, 2020.

[56] Serhan Gul, Sebastian Bosse, Dimitri Podborski, et al. Kalman Filter-based Head Motion Prediction for Cloud-based Mixed Reality. In *ACM International Conference on Multimedia (ACM MM)*, 2020.

[57] Fang-Yi Chao, Cagri Ozcinar, and Aljosa Smolic. Transformer-based Long-Term Viewport Prediction in 360deg Video: Scanpath is All You Need. In *IEEE International Workshop on Multimedia Signal Processing (MMSP)*, 2021.

[58] Ching-Ling Fan, Jean Lee, Wen-Chih Lo, et al. Fixation Prediction for 360deg Video Streaming in Head-Mounted Virtual Reality. In *nan*, 2017.

[59] Kaiyuan Hu, Haowen Yang, Yili Jin, et al. Understanding User Behavior in Volumetric Video Watching: Dataset, Analysis and Prediction. In *ACM International Conference on Multimedia (ACM MM)*, 2023.

[60] Yixuan Ban, Lan Xie, Zhimin Xu, et al. CUB360: Exploiting Cross-Users Behaviors for Viewport Prediction in 360 Video Adaptive Streaming. In *IEEE International Conference on Multimedia and Expo (ICME)*, 2018.

[61] Lan Xie, Xinggong Zhang, and Zongming Guo. CLS. In *ACM International Conference on Multimedia (ACM MM)*, 2018.

[62] Chenge Li, Weixi Zhang, Yong Liu, et al. Very Long Term Field of View Prediction for 360-Degree Video Streaming. In *IEEE Conference on Multimedia Information Processing and Retrieval (MIPR)*, 2019.

[63] Xianglong Feng, Yao Liu, and Wei Sheng. LiveDeep: Online Viewport Prediction for Live Virtual Reality Streaming Using Lifelong Deep Learning. In *IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*, 2020.

[64] Xianglong Feng, Zeyang Bao, and Sheng Wei. LiveObj: Object Semantics-based Viewport Prediction for Live Mobile Virtual Reality Streaming. *IEEE Transactions on Visualization and Computer Graphics*, 2021.

[65] Gazi Karam Illahi, Ashutosh Vaishnav, Teemu Kamarainen, et al. Learning to Predict Head Pose in Remotely-Rendered Virtual Reality. In *ACM Multimedia Systems Conference (MMSys)*, 2023.

[66] Zhiahao Zhang, Yiwei Chen, Weizhan Zhang, et al. Tile Classification Based Viewport Prediction with Multi-modal Fusion Transformer. In *ACM International Conference on Multimedia (ACM MM)*, 2023.

[67] Yanan Bao, Huasen Wu, Tianxiao Zhang, et al. Shooting a moving target: Motion-prediction-based transmission for 360-degree videos. In *IEEE International Conference on Big Data (BigData)*, 2016.

[68] Quentin Guimard, Lucile Sassatelli, Francesco Marchetti, et al. Deep variational learning for multiple trajectory prediction of 360deg head movements. In *ACM Multimedia Systems Conference (MMSys)*, 2022.

[69] Lan Xie, Zhimin Xu, Yixuan Ban, et al. 360ProbDASH. In *ACM International Conference on Multimedia (ACM MM)*, 2017.

[70] Ali Zeynali, Mohammad Hajiesmaili, and Ramesh K. Sitaraman. BOLA360. In *nan*, 2024.

[71] Jounsup Park, Philip A. Chou, and Jenq-Neng Hwang. Rate-Utility Optimized Streaming of Volumetric Media for Augmented Reality. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2019.

[72] Lisha Wang, Chenglin Li, Wenrui Dai, et al. QoE-Driven Adaptive Streaming for Point Clouds. *IEEE Transactions on Multimedia*, 2022.

[73] Yuanxing Zhang, Pengyu Zhao, Kaigui Bian, et al. DRL360: 360-degree Video Streaming with Deep Reinforcement Learning. In *nan*, 2019.

[74] Nuowen Kan, Junni Zou, Chenglin Li, et al. RAPT360: Reinforcement Learning-Based Rate Adaptation for 360-Degree Video Streaming With Adaptive Prediction and Tiling. *IEEE Transactions on Circuits and Systems for Video Technology*, 2021.

[75] Duo Wu, Panlong Wu, Miao Zhang, et al. MANSY: Generalizing Neural Adaptive Immersive Video Streaming With Ensemble and Representation Learning. *IEEE Transactions on Mobile Computing*, 2024.

[76] Lai Wei, Yanting Liu, Fangxin Wang, et al. FewVV: Few-Shot Adaptive Bitrate Volumetric Video Streaming With Prompted Online Adaptation. *IEEE Internet of Things Journal*, 2024.

[77] Jeroen van der Hooft, Tim Wauters, Filip De Turck, et al. Towards 6DoF HTTP Adaptive Streaming Through Point Cloud Compression. In *ACM International Conference on Multimedia (ACM MM)*, 2019.

[78] Jeremy Ouellette, Jashanjot Singh Sidhu, and Abdelhak Bentaleb. MazeLab: A Large-Scale Dynamic Volumetric Point Cloud Video Dataset With User Behavior Traces. In *ACM Multimedia Systems Conference (MMSys)*, 2025.

[79] Jeroen van der Hooft, Hadi Amirpour, Maria Torres Vega, et al. A Tutorial on Immersive Video Delivery: From Omnidirectional Video to Holography. *IEEE Communications Surveys & Tutorials*, 2023.

[80] Jack Jansen, Shishir Subramanyam, Romain Bouqueau, et al. A pipeline for multiparty volumetric video conferencing. In *ACM Multimedia Systems Conference (MMSys)*, 2020.

[81] Matthias De Fre, Jeroen van der Hooft, Tim Wauters, et al. Scalable MDC-Based Volumetric Video Delivery for Real-Time One-to-Many WebRTC Conferencing. In *nan*, 2024.

[82] Shishir Subramanyam, Irene Viola, Jack Jansen, et al. Evaluating the Impact of Tiled User-Adaptive Real-Time Point Cloud Streaming on VR Remote Communication. In *ACM International Conference on Multimedia (ACM MM)*, 2022.

[83] Anahita Mahzari, Afshin Taghavi Nasrabadi, Aliehsan Samiei, et al. FoV-Aware Edge Caching for Adaptive 360deg Video Streaming. In *ACM International Conference on Multimedia (ACM MM)*, 2018.

[84] Chamara Madarasingha, Kanchana Thilakarathna, and Albert Y. Zomaya. OpCASH: Optimized Utilization of MEC Cache for 360-Degree Video Streaming with Dynamic Tiling. In *IEEE International Conference on Pervasive Computing and Communications*, 2022.

[85] Niklas Ferdinand Carlsson and Derek L. Eager. Had You Looked Where I'm Looking? Cross-user Similarities in Viewing Behavior for 360-degree Video and Caching Implications. In *ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2020.

[86] Dongbiao He, Jinlei Jiang, Teng Ma, et al. CUBIST: High-Quality 360-Degree Video Streaming Services via Tile-based Edge Caching and FoV-Adaptive Prefetching. In *IEEE International Conference on Web Services (ICWS)*, 2021.

[87] Ziwen Ye, Qing Li, Xiaoteng Ma, et al. VRCT: A Viewport Reconstruction-Based 360deg Video Caching Solution for Tile-Adaptive Streaming. *IEEE Transactions on Broadcasting*, 2023.

[88] Yi-Zhen Tsai, James Luo, Yunshu Wang, et al. The World is Too Big to Download: 3D Model Retrieval for World-Scale Augmented Reality. In *ACM Multimedia Systems Conference (MMSys)*, 2023.

[89] Yanan Bao, Tianxiao Zhang, Amit Pande, et al. Motion-Prediction-Based Multicast for 360-Degree Video Transmissions. In *IEEE International Conference on Sensing, Communication, and Networking (SECON)*, 2017.

[90] MohammadJavad Salehi, Kari Hooli, Jari Hulkkonen, et al. Enhancing Next-Generation Extended Reality Applications With Coded Caching. *IEEE Open Journal of the Communications Society*, 2023.

[91] Anirudh Sabnis, Tareq Si Salem, Giovanni Neglia, et al. GRADES: Gradient Descent for Similarity Caching. In *nan*, 2021.

[92] Anahita Mahzari, Aliehsan Samiei, and Ravi Prakash. CooPEC: Cooperative Prefetching and Edge Caching for Adaptive 360deg Video Streaming. In *IEEE International Symposium on Multimedia (ISM)*, 2020.

[93] Nader Alfares and George Kesidis. Virtual Reality Benchmark

for Edge Caching Systems. In *IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*, 2025.

[94] Xavier Corbillon, Gwendal Simon, Alisa Devlic, et al. Viewport-adaptive navigable 360-degree video delivery. In *International Conference on Communications*, 2017.

[95] Mohammad Hosseini and Viswanathan Swaminathan. Adaptive 360 VR Video Streaming: Divide and Conquer. In *IEEE International Symposium on Multimedia (ISM)*, 2016.

[96] Liyang Sun, Fanyi Duanmu, Yong Liu, et al. Multi-path multitier 360-degree video streaming in 5G networks. In *ACM Multimedia Systems Conference (MMSys)*, 2018.

[97] Xuelin Liu, Haoyun Zhang, Jiebin Yan, et al. Quality of Experience of Viewport Adaptive Omnidirectional Video Streaming. In *Proceedings - International Conference on Image Processing*, 2024.

[98] Yuichiro Urata, Masanori Koike, Kazuhisa Yamagishi, et al. Extension of ITU-T P.1203 model to Tile-based Omnidirectional Video Streaming. *Electronic Imaging*, 2021.

[99] Masanori Koike, Yuichiro Urata, Noritsugu Egi, et al. Extension of ITU-T P.1204.3 Model to Tile-Based VR Streaming Services. In *IEEE International Workshop Technical Committee on Communications Quality and Reliability (CQR)*, 2021.

[100] Luyang Liu, Ruiguang Zhong, Wuyang Zhang, et al. Cutting the Cord. In *International Conference on Mobile Systems, Applications, and Services*, 2018.

[101] Zeqi Lai, Yang Hu, Yong Cui, et al. Furion: Engineering High-Quality Immersive Virtual Reality on Today's Mobile Devices. *IEEE Transactions on Mobile Computing*, 2019.

[102] Tiago M. Fernandez-Carames, Paula Fraga-Lamas, Manuel Suarez-Albela, et al. A Fog Computing and Cloudlet Based Augmented Reality System for the Industry 4.0 Shipyard. *Sensors*, 2018.

[103] Jinke Ren, Yinghui He, Guan Jia Huang, et al. An Edge-Computing Based Architecture for Mobile Augmented Reality. *IEEE Network*, 2019.

[104] Yu Wei Tan, Siang Ern Low, Jonas Chow, et al. DHR+S: distributed hybrid rendering with realistic real-time shadows for interactive thin client metaverse and game applications. *The Visual Computer*, 2024.

[105] Edward Lu, Sagar Bharadwaj, Mallesham Dasari, et al. RenderFusion: Balancing Local and Remote Rendering for Interactive 3D Scenes. In *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2023.

[106] Hongxin Zhang, Jin Zhang, Xue Yin, et al. Cloud-to-end Rendering and Storage Management for Virtual Reality in Experimental Education. *Virtual Reality & Intelligent Hardware*, 2020.

[107] Yu-Jung Chen, Chung-Yao Hung, and Shao-Yi Chien. Distributed rendering: Interaction delay reduction in remote rendering with client-end GPU-accelerated scene warping technique. In *IEEE International Conference on Multimedia & Expo Workshops (ICMEW)*, 2017.

[108] Joerg H. Mueller, Philip Voglreiter, Mark Dokter, et al. Shading atlas streaming. *ACM Transactions on Graphics*, 2018.

[109] Zhihui Ke, Xiaobo Zhou, Dadong Jiang, et al. CollabVr: Reprojection-Based Edge-Client Collaborative Rendering for Real-Time High-Quality Mobile Virtual Reality. In *Proceedings/Proceedings - Real-Time Systems Symposium*, 2023.

[110] Yong Li and Wei Gao. DeltaVR. In *International Conference on Information Processing in Sensor Networks (IPSN)*, 2019.

[111] Gazi Karam Illahi, Matti Siekkinen, Teemu Kamarainen, et al. Foveated streaming of real-time graphics. In *ACM Multimedia Systems Conference (MMSys)*, 2021.

[112] Rongji Liao, Yuan Zhang, Wei Zhang, et al. 3DGS-Enabled High-Fidelity Low-Cost Immersive Static 3D Video Streaming. *IEEE Journal on Selected Areas in Communications*, 2025.

[113] Zhihui Ke, Xiaobo Zhou, Yuyang Liu, et al. EdgeGaussian: Real-time Free-Viewpoint Video for Mobile VR via Edge-Client Collaborative Neural Rendering. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, 2025.

[114] He Zhu, Zheng Liu, Xingyang Li, et al. Nebula: Enable City-Scale 3D Gaussian Splatting in Virtual Reality via Collaborative Rendering and Accelerated Stereo Rasterization. *arXiv (Cornell University)*, 2025.

[115] Xukan Ran, Haolianz Chen, Xiaodan Zhu, et al. DeepDecision: A Mobile Deep Learning Framework for Edge Video Analytics. In *nan*, 2018.

[116] Wenxiao Zhang, Bo Han, and Pan Hui. SEAR: Scaling Experiences in Multi-user Augmented Reality. *IEEE Transactions on Visualization and Computer Graphics*, 2022.

[117] Z. Jonny Kong, Qiang Xu, and Yuanming Hu. ARISE: High-Capacity AR Offloading Inference Serving via Proactive Scheduling. In *International Conference on Mobile Systems, Applications, and Services*, 2024.

[118] Xing Chen and Guizhong Liu. Energy-Efficient Task Offloading and Resource Allocation via Deep Reinforcement Learning for Augmented Reality in Mobile Edge Networks. *IEEE Internet of Things Journal*, 2021.

[119] Peng Lin, Yan Liu, Zhizhong Zhang, et al. Cost-Aware Task Offloading and Migration for Wireless Virtual Reality Using Interactive A3C Approach. *IEEE Transactions on Vehicular Technology*, 2024.

[120] Qiang Liu, Siqi Huang, Johnson Opadere, et al. An Edge Network Orchestrator for Mobile Augmented Reality. In *nan*, 2018.

[121] Jie Ming Feng and Jun Zhao. Resource Allocation for Augmented Reality Empowered Vehicular Edge Metaverse. *IEEE Transactions on Communications*, 2023.

[122] Xinyu Zhou, Chang Liu, and Jun Zhao. Resource Allocation of Federated Learning for the Metaverse With Mobile Augmented Reality. *IEEE Transactions on Wireless Communications*, 2023.

[123] Mingzhe Chen, Walid Saad, and Changchuan Yin. Virtual Reality Over Wireless Networks: Quality-of-Service Model and Learning-Based Resource Management. *IEEE Transactions on Communications*, 2018.

[124] Ruitao Xie, Junhong Fang, Junmei Yao, et al. Sharing-Aware Task Offloading of Remote Rendering for Interactive Applications in Mobile Edge Computing. *IEEE Transactions on Cloud Computing*, 2021.

[125] Ruitao Xie, Junhong Fang, Junmei Yao, et al. QoS-Aware Scheduling of Remote Rendering for Interactive Multimedia Applications in Edge Computing. *IEEE Transactions on Parallel and Distributed Systems*, 2022.

[126] Kittipat Apicharttrisorn, Bharath Balasubramanian, Jiasi Chen, et al. Characterization of Multi-User Augmented Reality over Cellular Networks. In *IEEE International Conference on Sensing, Communication, and Networking (SECON)*, 2020.

[127] Moinak Ghoshal, Z. Jonny Kong, Qiang Xu, et al. Can 5G mmWave Enable Edge-Assisted Real-Time Object Detection for Augmented Reality? In *Proceedings - International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*, 2023.

[128] Kjell E. Brunnstrom, Elijs Dima, Tahir Naseer Qureshi, et al. Latency impact on Quality of Experience in a virtual reality simulator for remote control of machines. *Signal Processing Image Communication*, 2020.

[129] Abdullah S. Alshahrani, Ibrahim A. Elgendy, Ammar Muthanna, et al. Efficient Multi-Player Computation Offloading for VR Edge-Cloud Computing Systems. *Applied Sciences*, 2020.

[130] Kaifei Chen, Tong Li, Hyung-Sin Kim, et al. MARVEL. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2018.

[131] Yongjie Guan, Xueyu Hou, Nan Wu, et al. DeepMix. In *International Conference on Mobile Systems, Applications, and Services*, 2022.

[132] Federico Chiariotti, Matteo Drago, Paolo Testolina, et al. Temporal Characterization and Prediction of VR Traffic: A Network Slicing Use Case. *IEEE Transactions on Mobile Computing*, 2023.

[133] Eugene Korneev, Mikhail Liubogoshchev, Dmitry Bankov, et al. How to Model Cloud VR: An Empirical Study of Features That Matter. *IEEE Open Journal of the Communications Society*, 2024.

[134] Huanle Zhang, Ahmed Mustafa Elmokashfi, and Prasant Mohapatra. WiFi and Multiple Interfaces: Adequate for Virtual Reality? In *IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, 2018.

[135] Armin Becher, Jens Angerer, and Thomas Grauschopf. Novel

Approach to Measure Motion-To-Photon and Mouth-To-Ear Latency in Distributed Virtual Reality Systems. *arXiv (Cornell University)*, 2018.

[136] Ilja T. Feldstein and Stephen R. Ellis. A Simple Video-Based Technique for Measuring Latency in Virtual Reality or Teleoperation. *IEEE Transactions on Visualization and Computer Graphics*, 2020.

[137] Joohwan Kim, Pyarelal Knowles, Josef Spjut, et al. Post-Render Warp with Late Input Sampling Improves Aiming Under High Latency Conditions. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 2020.

[138] Viktor Kelkkanen, David Lindero, Markus Fiedler, et al. QoE of Frame Stalls in Remote 6-DOF VR. In *International Conference on Quality of Multimedia Experience (QoMEX)*, 2022.

[139] Teemu Kamarainen, Matti Siekkinen, Jukka Eerikainen, et al. CloudVR. In *ACM International Conference on Multimedia (ACM MM)*, 2018.

[140] Yen-Chun Li, Chia-Hsin Hsu, Yu-Chun Lin, et al. Performance Measurements on a Cloud VR Gaming Platform. In *nan*, 2020.

[141] Henrique Souza Rossi, Karan Mitra, Samuel Larsson, et al. Subjective QoE Assessment for Virtual Reality Cloud-based First-Person Shooter Game. In *International Conference on Communications*, 2024.

[142] Steven Gao, Jeffrey Liu, Qinjun Jiang, et al. XRgo: Design and Evaluation of Rendering Offload for Low-Power Extended Reality Devices. In *ACM Multimedia Systems Conference (MMSys)*, 2025.

[143] Thanh Cong Nguyen, Sanghyun Kim, Ju-Hyung Son, et al. Selective Timewarp Based on Embedded Motion Vectors for Interactive Cloud Virtual Reality. *IEEE Access*, 2018.

[144] Xueshi Hou and Sujit Dey. Motion Prediction and Pre-Rendering at the Edge to Enable Ultra-Low Latency Mobile 6DoF Experiences. *IEEE Open Journal of the Communications Society*, 2020.

[145] Abbas Mehrabi, Matti Siekkinen, Teemu Kamarainen, et al. Multi-Tier CloudVR. *ACM Transactions on Multimedia Computing Communications and Applications*, 2021.

[146] Luyang Liu, Hongyu Li, and Marco Gruteser. Edge Assisted Real-time Object Detection for Mobile Augmented Reality. In *nan*, 2019.

[147] Wenxiao Zhang, Sikun Lin, Farshid Hassani Bijarbooneh, et al. CloudAR. In *nan*, 2017.

[148] Wenxiao Zhang, Bo Han, and Pan Hui. Jaguar. In *ACM International Conference on Multimedia (ACM MM)*, 2018.

[149] Maximilian Warsinke, Tanja Kojic, Maurizio Vergari, et al. VR Cloud Gaming UX: Exploring the Impact of Network Quality on Emotion, Presence, Game Experience and Cybersickness. In *IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, 2024.

[150] Ali Al-Shuwaili and Osvaldo Simeone. Energy-Efficient Resource Allocation for Mobile Edge Computing-Based Augmented Reality Applications. *IEEE Wireless Communications Letters*, 2017.

[151] Christoph Bachhuber, Alvaro Sanchez Martinez, Rastin Pries, et al. Edge Cloud-based Augmented Reality. In *IEEE International Workshop on Multimedia Signal Processing (MMSP)*, 2019.

[152] Baraka William Nyamtiga, Airlangga Adi Hermawan, Yakub Fahim Luckyarno, et al. Edge-Computing-Assisted Virtual Reality Computation Offloading: An Empirical Study. *IEEE Access*, 2022.

[153] Matthijs Jansen, Jesse Donkervliet, Animesh Trivedi, et al. Can My WiFi Handle the Metaverse? A Performance Evaluation Of Meta's Flagship Virtual Reality Hardware. In *ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2023.

[154] Lorenzo Romagnoli, Gabriele Proietti Mattia, and Roberto Beraldi. A Study on Energy Efficiency in Edge-assisted VR Applications with Meta Quest 2 for Disaster Management. In *International Conference on Information and Communication Technologies for Disaster Management (ICT-DM)*, 2023.

[155] Christina Chaccour, Mehdi Naderi Soorki, Walid Saad, et al. Can Terahertz Provide High-Rate Reliable Low-Latency Communications for Wireless VR? *IEEE Internet of Things Journal*, 2022.

[156] Anfu Zhou, Leilei Wu, Shaoqing Xu, et al. Following the Shadow: Agile 3-D Beam-Steering for 60 GHz Wireless Networks. In *nan*, 2018.

[157] Jakob Struye, Sam Van Damme, Nabeel Nisar Bhat, et al. Toward Interactive Multi-User Extended Reality Using Millimeter-Wave Networking. *IEEE Communications Magazine*, 2024.

[158] Rizqi Hersyandika, Haoqiu Xiong, Yang Miao, et al. Movement-Robust mmWave VR via Dual-Beam Reception and Predictive Beam Transition. *IEEE Open Journal of the Communications Society*, 2025.

[159] Arno Moerman, Olivier Caytan, Hendrik Rogier, et al. Millimeter-Wave Distributed Antenna Systems for Interactive Virtual Reality: Reducing blockage and exposure and enhancing robustness with (air-filled) substrate-integrated waveguide antennas. [Bioelectromagnetics]. *IEEE Antennas and Propagation Magazine*, 2025.

[160] Jakob Struye, Filip Lemic, and Jeroen Famaey. CoVRage: Millimeter-Wave Beamforming for Mobile Interactive Virtual Reality. *IEEE Transactions on Wireless Communications*, 2022.

[161] Lihao Zhang, Paul Kudyba, Zhenlin An, et al. RadioSight: Predictive mmWave XR Network Optimization from Dynamic Neural Radio Fields. *arXiv (Cornell University)*, 2026.

[162] Xiaonan Liu, Yansha Deng, Chong Zhao Han, et al. Learning-Based Prediction, Rendering and Transmission for Interactive Virtual Reality in RIS-Assisted Terahertz Networks. *IEEE Journal on Selected Areas in Communications*, 2021.

[163] Rui Huang, Vincent W. S. Wong, and Robert Schober. Rate-Splitting for Intelligent Reflecting Surface-Aided Multiuser VR Streaming. *IEEE Journal on Selected Areas in Communications*, 2023.

[164] Jakob Struye, Filip Lemic, and Jeroen Famaey. Towards Ultra-Low-Latency mmWave Wi-Fi for Multi-User Interactive Virtual Reality. In *Globecom*, 2020.

[165] Molham Alsakati, Charlie Pettersson, Sebastian Max, et al. Performance of 802.11be Wi-Fi 7 with Multi-Link Operation on AR Applications. In *IEEE Wireless Communications and Networking Conference*, 2023.

[166] Vyacheslav Loginov, Sergei Tutelian, Ivan Startsev, et al. Scalability of Wi-Fi Performance in Virtual Reality Scenarios. *Sensors*, 2025.

[167] Costas Michaelides, Miguel Casasnovas, Daniele Marchitelli, et al. Is Wi-Fi 6 Ready for Virtual Reality Mayhem? A Case Study Using One AP and Three HMDs. In *IEEE Future Networks World Forum (FNWF)*, 2023.

[168] Marc Carrascosa, Lorenzo Galati Giordano, Francesc Wilhelmi, et al. Performance Evaluation of MLO for XR Streaming: Can Wi-Fi 7 Meet the Expectations? In *IEEE International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)*, 2024.

[169] Pouria Paymard, Abolfazl Amiri, Troels Kolding, et al. Enhanced Link Adaptation for Extended Reality Code Block Group based HARQ Transmissions. In *IEEE Globecom Workshops (GC Wkshps)*, 2022.

[170] Yaping Sun, Zhiyong Chen, Meixia Tao, et al. Communications, Caching, and Computing for Mobile Virtual Reality: Modeling and Tradeoff. *IEEE Transactions on Communications*, 2019.

[171] Tian Dang and Mugen Peng. Joint Radio Communication, Caching, and Computing Design for Mobile Virtual Reality Delivery in Fog Radio Access Networks. *IEEE Journal on Selected Areas in Communications*, 2019.

[172] Cristina Perfecto, Mohammed S. Elbamby, Javier Del Ser, et al. Taming the Latency in Multi-User VR 360deg: A QoE-Aware Deep Learning-Aided Multicast Framework. *IEEE Transactions on Communications*, 2020.

[173] Yuan Zhang, Lingjun Pu, Tao Lin, et al. QoE-Oriented Mobile Virtual Reality Game in Distributed Edge Networks. *IEEE Transactions on Multimedia*, 2023.

[174] Jiangong Chen, Feng Qian, and Bin Li. Enhancing Quality of Experience for Collaborative Virtual Reality with Commodity Mobile Devices. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2022.

[175] Nam H. Chu, Dinh Thai Hoang, Diep Ngoc Nguyen, et al. MetaSlicing: A Novel Resource Allocation Framework for Metaverse. *IEEE Transactions on Mobile Computing*, 2023.

[176] Cheng-Yeh Chen and Hung-Yun Hsieh. Cross-Frame Resource

Allocation With Context-Aware QoE Estimation for 360deg Video Streaming in Wireless Virtual Reality. *IEEE Transactions on Wireless Communications*, 2023.

[177] Andreas Casparsen, Beatriz Soret, Jimmy Jessen Nielsen, et al. Near Real-Time Data-Driven Control of Virtual Reality Traffic in Open Radio Access Network. In *Globecom*, 2023.

[178] Serhan Gul, Saba Ahsan, Stefano Paris, et al. PDU Set-Based QoS Handling in 3GPP: Release 18 Overview and Future Directions. *IEEE Communications Standards Magazine*, 2025.

[179] Moyukh Laha and Gabriel-Miro Muntean. Protocol Data Unit Set-based QoS-aware Downlink Scheduling for Extended Reality Applications in 6G. In *International Conference on Communications*, 2026.

[180] Dongru Li, Huazheng You, Wei Yong Jiang, et al. Enhanced Power Saving Schemes for eXtended Reality. In *IEEE International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, 2021.

[181] Andreas Casparsen, Beatriz Soret, Jimmy Jessen Nielsen, et al. Near Real-Time Data-Driven Adaptive RAN Slicing of VR Traffic in ORAN. *IEEE Open Journal of the Communications Society*, 2025.

[182] Margarita Gapeyenko, Stefano Paris, Markus Isomaki, et al. Overview of NR Enhancements for Extended Reality (XR) in 3GPP 5G-Advanced. *IEEE Access*, 2024.

[183] Conghao Zhou, Jie Gao, Shisheng Hu, et al. User-Centric Communication Service Provision for Edge-Assisted Mobile Augmented Reality. *IEEE Transactions on Mobile Computing*, 2025.

[184] Hongyang Du, Jiazhen Liu, Dusit Tao Niyato, et al. Attention-Aware Resource Allocation and QoE Analysis for Metaverse xURLLC Services. *IEEE Journal on Selected Areas in Communications*, 2023.

[185] Lei Feng, Xiaoyi Jiang, Yao Sun, et al. Resource Allocation for Metaverse Experience Optimization: A Multi-Objective Multi-Agent Evolutionary Reinforcement Learning Approach. *IEEE Transactions on Mobile Computing*, 2024.

[186] Panagiotis Nikolaidis, Samie Mostafavi, James Gross, et al. A Proof of Concept Resource Management Scheme for Augmented Reality Applications in 5G Systems. In *IEEE International Symposium on Dynamic Spectrum Access Networks*, 2025.

[187] Turan Eminli. *Cost Optimization of Sustainable and Elastic Cloud Architecture for QoE-aware XR applications*. PhD thesis, KTH Publication Database DiVA (KTH Royal Institute of Technology), 2025.

[188] Omar Eltobgy, Omar Arafa, and Mohamed M. Hefeeda. Mobile Streaming of Live 360-Degree Videos. *IEEE Transactions on Multimedia*, 2020.

[189] Yu Liu, Puqi Zhou, Zejun Zhang, et al. MuV2: Scaling up Multi-user Mobile Volumetric Video Streaming via Content Hybridization and Sharing. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, 2024.

[190] Minrui Xu, Dusit Tao Niyato, Jiawen Kang, et al. Wireless Edge-Empowered Metaverse: A Learning-Based Incentive Mechanism for Virtual Reality. In *International Conference on Communications*, 2022.

[191] Matthias De Fre, Jeroen van der Hooft, Chia-Yu Chang, et al. Low-Latency Volumetric Video Conferencing in Congested Networks Through L4S. In *ACM Multimedia Systems Conference (MMSys)*, 2025.

[192] Jangwoo Son, Yago Sanchez, Cornelius Hellge, et al. Split Rendering with L4S Over 5G for Latency Critical Interactive XR Applications. *IEEE Communications Magazine*, 2024.

[193] Vinicius Rodrigues Oliveira, Daniel Henriques Cezar Miranda Soares, and Daniel Fernandes Macedo. Characterizing the Network Traffic of Virtual Reality Cloud Gaming Applications. In *nan*, 2026.

[194] Daniel Mejias, Inhar Yeregui, Angel Martin, et al. Streaming Remote rendering services: Comparison of QUIC-based and WebRTC Protocols. *arXiv (Cornell University)*, 2025.

[195] Matthias De Fre, Jeroen van der Hooft, Jack Jansen, et al. WebRTC-Based Volumetric Video Conferencing: SFU Architecture Evaluation and Benchmarking. In *nan*, 2026.

[196] Miguel Casasnovas, Costas Michaelides, Marc Carrascosa, et al. Experimental evaluation of interactive Edge/Cloud Virtual Reality gaming over Wi-Fi using unity render streaming. *Computer Communications*, 2024.

[197] Ferran Maura, Francesc Wilhelmi, and Boris Bellalta. Evaluating Wi-Fi Performance for VR Streaming: A Study on Realistic HEVC Video Traffic. In *nan*, 2025.

[198] Yuhong Huang, Lei Cao, Hongbiao Zhang, et al. Model Between Network Quality and XR User's Experience in Terms of Video Stalling for Cloud Rendering XR. *IEEE Access*, 2025.

[199] Muhammad Zawish, Fayaz Ali Dharejo, Sunder Ali Khowaja, et al. AI and 6G Into the Metaverse: Fundamentals, Challenges and Future Research Trends. *IEEE Open Journal of the Communications Society*, 2024.

[200] A. Kak, Huu-Trung Thieu, Van-Quan Pham, et al. AweRAN. In *nan*, 2023.

[201] Luyi Chang, Zhe Zhang, Pei Li, et al. 6G-Enabled Edge AI for Metaverse: Challenges, Methods, and Future Research Directions. *Journal of Communications and Information Networks*, 2022.

[202] Margarita Gapeyenko, Vitaly Petrov, Stefano Paris, et al. Standardization of Extended Reality (XR) over 5G and 5G-Advanced 3GPP New Radio. *IEEE Network*, 2023.

[203] Prashanth H. Hande, Peerapol Tinnakornsrisuphap, Jelena Damnjanovic, et al. Extended Reality Over 5G–Standards Evolution. *IEEE Journal on Selected Areas in Communications*, 2023.

[204] Ruizhi Cheng, Nan Wu, Matteo Varvello, et al. Are we ready for metaverse? In *ACM Internet Measurement Conference (IMC)*, 2022.

[205] Ruizhi Cheng, Nan Wu, Songqing Chen, et al. Will Metaverse Be NextG Internet? Vision, Hype, and Reality. *IEEE Network*, 2022.

[206] Mallesham Dasari, Edward Lu, Michael W. Farb, et al. Scaling VR Video Conferencing. In *IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*, 2023.

[207] Kai Li, Yingping Cui, Weicai Li, et al. When Internet of Things Meets Metaverse: Convergence of Physical and Cyber Worlds. *IEEE Internet of Things Journal*, 2022.

[208] Fangyu Lin, Yingdong Hu, Zhening Liu, et al. Mon3tr: Monocular 3D Telepresence with Pre-built Gaussian Avatars as Amortization. *arXiv (Cornell University)*, 2026.

[209] Xiao Wen Xu, Siyuan Zhang, Qian Liu, et al. QoE-driven Delay-adaptive Control Scheme Switching for Time-delayed Bilateral Teleoperation with Haptic Data Reduction. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

[210] Thien Huynh-The, Thippa Reddy Gadekallu, Weizheng Wang, et al. Blockchain for the metaverse: A Review. *Future Generation Computer Systems*, 2023.

[211] Stephen Siemonsma and Tyler Bell. HoloKinect: Holographic 3D Video Conferencing. *Sensors*, 2022.

[212] Aditya Dhakal, Xukan Ran, Yunshu Wang, et al. SLAM-share. In *ACM International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2022.

[213] Aida Vidal-Balea, Oscar Blanco-Novoa, Paula Fraga-Lamas, et al. Off-Cloud Anchor Sharing Framework for Multi-User and Multi-Platform Mixed Reality Applications. *Applied Sciences*, 2025.

[214] Md Tanjemul Islam, Mohammad Shafin, and Md Rafiul Kabir. Seeing What the Vehicle Sees: Video-Augmented Virtual Reality for Physical Autonomous Vehicles. *arXiv (Cornell University)*, 2026.

[215] Dragos-Alexandru Cazacu, Matei Dragutu, Mihail Hanga, et al. Real-Time Interaction and Monitoring of Collaborative Robots via AR/VR and Digital Twin in IIoT Frameworks. *International Journal of Modeling and Optimization*, 2026.

[216] Adnan Aijaz, Mischa Dohler, A. Hamid Aghvami, et al. Realizing the Tactile Internet: Haptic Communications over Next Generation 5G Cellular Networks. *IEEE Wireless Communications*, 2016.

[217] Sam Van Damme, Javad Sameri, Susanna Schwarzmann, et al. Impact of Latency on QoE, Performance, and Collaboration in Interactive Multi-User Virtual Reality. *Applied Sciences*, 2024.

[218] Torin Hopkins, Suibi Che-Chuan Weng, Rishi Vanukuru, et al. Studying the Effects of Network Latency on Audio-Visual Perception During an AR Musical Task. In *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2022.

[219] Chuanqi Zheng, Annalisa Jarecki, and Kiju Lee. Integrated system architecture with mixed-reality user interface for virtual-physical hybrid swarm simulations. *Scientific Reports*, 2023.

[220] Nikolai Suslov. Implementing Decentralized Virtual Time in P2P Collaborative Learning Environment for Web XR. In *International Conference of the Immersive Learning Research Network (iLRN)*, 2021.

[221] George Kokiadis, Antonis I. Protopsaltis, Michalis Morfiadakis, et al. Decoupled Edge Physics Algorithms for Collaborative XR Simulations. *Computer Animation and Virtual Worlds*, 2024.

[222] Claudio Marche, Michele Nitti, Luigi Atzori, et al. Bridging IoT and the Metaverse: Policies for the Synchronization of Digital Twins. In *International Conference on Communications*, 2026.

[223] Tianyuan Yu, Xinyu Ma, Varun Patil, et al. Exploring the Design of Collaborative Applications via the Lens of NDN Workspace. In *nan*, 2024.

[224] Pei Ren, Xiuquan Qiao, Yakun Huang, et al. Edge AR X5: An Edge-Assisted Multi-User Collaborative Framework for Mobile Web Augmented Reality in 5G and Beyond. *IEEE Transactions on Cloud Computing*, 2020.

[225] Yuqi Zhou, Shuqi Liao, Yufeng Chen, et al. Scalable Collocated Multi-User VR Through Virtual Environment User Spatial Coherence. *IEEE Transactions on Visualization and Computer Graphics*, 2026.

[226] Volodymyr Duchenchuk and Volodymyr Boublik. Consistency Models for Distributed Virtual Reality Systems. *NaUKMA Research Papers Computer Science*, 2020.

[227] Felix G. Hamza-Lup and Jannick P. Rolland. Scene Synchronization for Real-Time Interaction in Distributed Mixed Reality and Virtual Reality Environments. *arXiv (Cornell University)*, 2018.

[228] Taehyun James Rhee, Stephen Thompson, Daniel Medeiros, et al. Augmented Virtual Teleportation for High-Fidelity Telecollaboration. *IEEE Transactions on Visualization and Computer Graphics*, 2020.

[229] Shuqing Li, C. Zhang, Binchang Li, et al. When Shared Worlds Break: Demystifying Defects in Multi-user Extended Reality Software Systems. *Proceedings of the ACM on software engineering.*, 2026.

[230] Marc Erich Latoschik, Florian Kern, Jan-Philipp Stauffert, et al. Not Alone Here?! Scalability and User Experience of Embodied Ambient Crowds in Distributed Social Virtual Reality. *IEEE Transactions on Visualization and Computer Graphics*, 2019.

[231] Umer Ijaz, Arslan Dawood Butt, Muhammad Husnain Khalid, et al. A low latency quaternion-based web transmission system for augmented reality applications. *NUST Journal of Engineering Sciences*, 2018.

[232] Kyung-Kyu Kang, Cho Rong Yu, Youn Hee Gil, et al. Metaverse Concert Platform for Live Performance and Large-Scale Audience Interaction. In *International Conference on Information and Communication Technology Convergence (ICTC)*, 2025.

[233] Xukan Ran, Carter Slocum, Yi-Zhen Tsai, et al. Multi-user augmented reality with communication efficient and spatially consistent virtual objects. In *ACM International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2020.

[234] Zhiyong Chen, Haoyu Zhu, Li Song, et al. Wireless Multiplayer Interactive Virtual Reality Game Systems With Edge Computing: Modeling and Optimization. *IEEE Transactions on Wireless Communications*, 2022.

[235] Haoyu Zhu, Yingjiao Li, Zhiyong Chen, et al. Mobile Edge Resource optimization for Multiplayer Interactive Virtual Reality Game. In *IEEE Wireless Communications and Networking Conference*, 2021.

[236] Robin Sidhu, Xiang Su, Xiaoli Liu, et al. Towards Scalable, Low-Latency Volumetric Streaming: A Hybrid Predictive Synchronization Framework. In *International Conference on Communications*, 2026.

[237] John O Murray, Yasra Chandio, and Michael Zink. Exploring the Relationship Between Quality of Experience and Quality of Service in Collaborative Mixed Reality. In *ACM International Conference on Interactive Media Experiences*, 2026.

[238] Bowen Qin, Kurt Tutschku, and Yan Hu. Requirements for network resource management in multi-user extended reality systems: a systematic review. *Frontiers in Communications and Networks*, 2026.

[239] Ruizhi Cheng, Nan Wu, Matteo Varvello, et al. A First Look at Immersive Telepresence on Apple Vision Pro. In *nan*, 2024.

[240] Sergi Fernandez, Mario Montagud, David Rincon, et al. Addressing Scalability for Real-time Multiuser Holo-portation: Introducing and Assessing a Multipoint Control Unit (MCU) for Volumetric Video. In *ACM International Conference on Multimedia (ACM MM)*, 2023.

[241] Atanas Vlahov, Vladimir K. Poulkov, and Albena Mihovska. Analysis of Open RAN Performance Indicators Related to Holographic Telepresence Communications. In *International Symposium on Wireless Personal Multimedia Communications (WPMC)*, 2021.

[242] Nadir Weibel, Ben Alwood, Vishwajith Ramesh, et al. HoloStroke: Assessing for Immersive Stroke Care Through Stroke Hologram Teleportation. *Telemedicine Journal and e-Health*, 2024.

[243] William Menz, Alexander Zoubarev, David Kutschke, et al. Evaluation of a Floating-Head Communication Prototype for Video-Conferencing. In *International Symposium on Multimedia*, 2025.

[244] Ashutosh Singla, Irene Viola, Jack Jansen, et al. QoE Evaluation of Remote Physiotherapy in Volumetric Video and Video-Based Real-Time Communication. In *IEEE International Conference on Multimedia and Expo (ICME)*, 2025.

[245] Gulnaziye Bingol, Lazizjon Suyunov, Zukhriddin Kamolov, et al. QoE in Multi-user Collaborative Virtual Reality Games: Impact of Network and Avatar Quality. In *International Conference on Quality of Multimedia Experience (QoMEX)*, 2025.

[246] Changyang She, Chenyang Yang, and Tony Q. S. Quek. Cross-Layer Transmission Design for Tactile Internet. In *Globecom*, 2016.

[247] Xiao Wen Xu, Qian Liu, and Eckehard G. Steinbach. Toward QoE-driven dynamic control scheme switching for time-delayed teleoperation systems: A dedicated case Study. In *IEEE International Symposium on Haptic, Audio and Visual Environments and Games (HAVE)*, 2017.

[248] Burak Kizilkaya, Changyang She, Guodong Zhao, et al. Task-Oriented Prediction and Communication Co-Design for Haptic Communications. *IEEE Transactions on Vehicular Technology*, 2023.

[249] Mads Antonsen, Francesco Chinello, and Qi Zhang. Design Octree-Based Method to Improve Model-Mediated Teleoperation in Tactile Internet. In *Proceedings - IEEE International Conference on Robotics and Automation/Proceedings*, 2024.

[250] Ekaterina Ivanova, Jonathan Eden, Silei Zhu, et al. Short Time Delay Does Not Hinder Haptic Communication Benefits. *IEEE Transactions on Haptics*, 2021.

[251] Marian Ulbricht, Stefan Senk, Hosein Kangavar Nazari, et al. TSN-FlexTest: Flexible TSN Measurement Testbed. *IEEE Transactions on Network and Service Management*, 2023.

[252] Vineet Gokhale, Mohamad A. Eid, Kees Kroep, et al. Toward Enabling High-Five Over WiFi: A Tactile Internet Paradigm. *IEEE Communications Magazine*, 2021.

[253] Qianqian Tong, Wenxuan Wei, Congying Liu, et al. Cross-Modal Transmission with Active Packet Loss and Restoration for Tactile Internet. *IEEE Communications Magazine*, 2024.

[254] Premankur Banerjee, Jiaxuan Wang, Lauren Tomita, et al. Virtual Encounters of the Haptic Kind: Towards a Multi-User VR System for Real-Time Social Touch. In *IEEE World Haptics Conference (WHC)*, 2025.

[255] Jie Cao, Xu Zhu, Sumei Sun, et al. Toward Industrial Metaverse: Age of Information, Latency and Reliability of Short-Packet Transmission in 6G. *IEEE Wireless Communications*, 2023.

[256] Omar Hashash, Christina Chaccour, Walid Saad, et al. Towards a Decentralized Metaverse: Synchronized Orchestration of Digital Twins and Sub-Metaverses. In *International Conference on Communications*, 2023.

[257] Zhen Meng, Kan Chen, Yufeng Diao, et al. Task-Oriented Cross-System Design for Timely and Accurate Modeling in the Metaverse. *IEEE Journal on Selected Areas in Communications*, 2023.

[258] Tarinee Tonggoed, Panu Leelanuch, Suttiphong Sridokbuab, et al. IoT-Enabled VR Simulation Platform for Cobot Motion Visualization and Remote Control. In *International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, 2025.

[259] Lai-Sum Yim, Quang TN Vo, Ching-I Huang, et al. WFH-VR: Teleoperating a Robot Arm to set a Dining Table across the Globe via Virtual Reality. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[260] Chunyang Zhao, Zeyu Zhou, Haoran Liu, et al. Towards Kbps-level Vehicle Teleoperation via Persistent-Transient Environment Modelling. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[261] Cephas B. Mwimangire, Mohammad Hosseingholizadeh, Hamidreza Ostadabbas, et al. Design and Implementation of an AR System for Real-Time Urban Model Editing and Visualization. *The international archives of the photogrammetry, remote sensing and spatial information sciences/International archives of the photogrammetry, remote sensing and spatial information sciences*, 2026.

[262] Karan Mitra, Henrique Souza Rossi, Justin Gavrell, et al. QoE Assessment of Cloud-Based Social Extended Reality Applications Over Heterogeneous Access Networks. In *nan*, 2025.

[263] Fenghe Hu, Yansha Deng, Walid Saad, et al. Cellular-Connected Wireless Virtual Reality: Requirements, Challenges, and Solutions. *IEEE Communications Magazine*, 2020.

[264] Carlos Cortes, Pablo Perez, Jesus Gutierrez, et al. Influence of Video Delay on Quality, Presence, and Sickness in Viewport Adaptive Immersive Streaming. In *International Conference on Quality of Multimedia Experience (QoMEX)*, 2020.

[265] Minh Nguyen, Shivi Vats, Sam Van Damme, et al. Characterization of the Quality of Experience and Immersion of Point Cloud Videos in Augmented Reality Through a Subjective Study. *IEEE Access*, 2023.

[266] Anouk van Kasteren, Kjell E. Brunnstrom, John Hedlund, et al. Quality of experience of 360 video - subjective and eye-tracking assessment of encoding and freezing distortions. *Multimedia Tools and Applications*, 2022.

[267] Jannis Weil, Yassin Alkhalili, Anam Tahir, et al. Modeling Quality of Experience for Compressed Point Cloud Sequences based on a Subjective Study. In *International Conference on Quality of Multimedia Experience (QoMEX)*, 2023.

[268] Tanja Kojic, Steven Schmidt, Sebastian Moller, et al. Influence of Network Delay in Virtual Reality Multiplayer Exergames: Who is actually delayed? In *International Conference on Quality of Multimedia Experience (QoMEX)*, 2019.

[269] Ahmad Yousef Alhilal, Kirill Shatilov, Gareth Tyson, et al. Network Traffic in the Metaverse: The Case of Social VR. In *IEEE International Conference on Distributed Computing Systems Workshops (ICDCSW)*, 2023.

[270] Henrique Souza Rossi, Karan Mitra, Christer Ahlund, et al. QoE Models for Virtual Reality Cloud-based First Person Shooter Game over Mobile Networks. In *International Conference on Network and Service Management (CNSM)*, 2024.

[271] Huyen T. T. Tran, Nam Pham Ngoc, Cuong Manh Bui, et al. An evaluation of quality metrics for 360 videos. In *International Conference on Ubiquitous and Future Networks*, 2017.

[272] Stephan Fremerey, Steve Goring, Rakesh Rao Ramachandra Rao, et al. Subjective Test Dataset and Meta-data-based Models for 360deg Streaming Video Quality. In *IEEE International Workshop on Multimedia Signal Processing (MMSP)*, 2020.

[273] Kuan-Yu Lee, Ashutosh Singla, Pablo S. Cesar, et al. Adaptive Cloud VR Gaming Optimized by Gamer QoE Models. *ACM Transactions on Multimedia Computing Communications and Applications*, 2024.

[274] Md Tariqul Islam, Christian Esteve Rothenberg, and Pedro Henrique Gomes. Predicting XR Services QoE with ML: Insights from In-band Encrypted QoS Features in 360-VR. In *IEEE International Conference on Network Softwarization (NetSoft)*, 2023.

[275] Francisco Germano Vogt, Fabricio Rodriguez, Ariel Goes de-Castro, et al. QoEyes: Towards Virtual Reality Streaming QoE Estimation Entirely in the Data Plane. In *IEEE International Conference on Network Softwarization (NetSoft)*, 2023.

[276] Georgios Kougioumtzidis, Atanas Vlahov, Vladimir K. Poulkov, et al. Deep Learning-Aided QoE Prediction for Virtual Reality Applications Over Open Radio Access Networks. *IEEE Access*, 2023.

[277] Roberto Iraja Tavares da Costa Filho, Marcelo Caggiani Luizelli, Maria Torres Vega, et al. Predicting the performance of virtual reality video streaming in mobile networks. In *ACM Multimedia Systems Conference (MMSys)*, 2018.

[278] Pablo Perez, Ester Gonzalez-Sosa, Jesus Gutierrez, et al. Emerging Immersive Communication Systems: Overview, Taxonomy, and Good Practices for QoE Assessment. *Frontiers in Signal Processing*, 2022.

[279] Sihao Zhao, Hatem Abou-Zeid, Ramy Atawia, et al. Virtual Reality Gaming on the Cloud: A Reality Check. In *Globecom*, 2021.

[280] Mattia Lecci, Matteo Drago, Andrea Zanella, et al. An Open Framework for Analyzing and Modeling XR Network Traffic. *IEEE Access*, 2021.

[281] Yihang Zhang, Zhidong Jia, Li Jiang, et al. Modeling Virtual Reality Traffic with Head Movement in Remote Rendering. In *International Conference on Communications*, 2025.

[282] Tiziana Cattai, Luca Mastrandrea, Alessandro Priviero, et al. Glancing at Extended Reality: An Empirical Model of 3D Animated XR Data Traffic. In *IFIP Networking Conference (IFIP Networking)*, 2024.

[283] Gabriel Meyer-Lee, Jiacheng Shang, and Jie Wu. Location-leaking through Network Traffic in Mobile Augmented Reality Applications. In *IEEE International Performance, Computing, and Communications Conference*, 2018.

[284] Sheikh Samit Muhaimin and Spyridon Mastorakis. Identifying VR User Activity: A Network Traffic Analysis Approach. In *nan*, 2025.

[285] Zihao Su, Kunlin Cai, Reuben Beeler, et al. Remote Keylogging Attacks in Multi-user VR Applications. *arXiv (Cornell University)*, 2024.

[286] Abdullah Al Arafat, Zhishan Guo, and Amro Awad. VR-Spy: A Side-Channel Attack on Virtual Key-Logging in VR Headsets. In *nan*, 2021.

[287] Wei Xing, Chenyang Yang, and Chengjian Sun. Privacy Leakage in Proactive VR Streaming: Modeling and Tradeoff. *arXiv (Cornell University)*, 2022.

[288] Naureen Hoque. What AR Glasses Leak: Physical Scene Inference from Encrypted Traffic. In *nan*, 2026.

[289] Rahmadi Trimananda, Hieu Le, Hao Cui, et al. OVRseen: Auditing Network Traffic and Privacy Policies in Oculus VR. *arXiv (Cornell University)*, 2021.

[290] Zuo Xiang, Frank Gabriel, Elena Urbano, et al. Reducing Latency in Virtual Machines: Enabling Tactile Internet for Human-Machine Co-Working. *IEEE Journal on Selected Areas in Communications*, 2019.

[291] Mohammad Waquas Usmani, Susmit Shannigrahi, and Michael Zink. ABE-VVS: Attribute-Based Encrypted Volumetric Video Streaming. *arXiv (Cornell University)*, 2026.

[292] Raghad Hazim Saeed, Qutaiba I. Ali, and Farhad E. Mahmood. Secure Virtual Reality Streaming Under Attack: Fine-Grained Modeling of the Latency Security Trade-Off. *International Journal of Safety and Security Engineering*, 2026.

[293] Xing Wei and Chenyang Yang. FoV Privacy-aware VR Streaming. In *IEEE Wireless Communications and Networking Conference*, 2022.

[294] Xing Wei, Cornelius Hellge, Chenyang Yang, et al. Improving QoE-Privacy Tradeoff in XR Streaming. *IEEE Signal Processing Letters*, 2024.

[295] Sheyda Mirzakhani and Niklas Ferdinand Carlsson. Priv360: Application-Oriented QoE-Optimized Client-Side Protection for 360-Viewer Identification. *Proceedings on Privacy Enhancing Technologies*, 2026.

[296] Jongseok Ryu, Seunghwan Son, Joonyoung Lee, et al. Design of Secure Mutual Authentication Scheme for Metaverse Environments Using Blockchain. *IEEE Access*, 2022.

[297] Ismaila Adeniyi Kamil and Sunday Oyinlola Ogundoyin. A lightweight mutual authentication and key agreement protocol for remote surgery application in Tactile Internet environment. *Computer Communications*, 2021.

[298] Ruizhi Cheng, Songqing Chen, and Bo Han. Toward Zero-Trust Security for the Metaverse. *IEEE Communications Magazine*, 2023.

[299] Tarek Gaber, Joseph Bamidele Awotunde, Mohamed F. Torky,

et al. Metaverse-IDS: Deep learning-based intrusion detection system for Metaverse-IoT networks. *Internet of Things*, 2023.

[300] Beyza Butun, Aristide Tanyi-Jong Akem, Michele Gucciardo, et al. Fast Detection of Cyberattacks on the Metaverse through User-plane Inference. In *nan*, 2023.

[301] Sandeep Kumar Jangir, Avinash Awasthi, Pritam Vediya, et al. TTDF-MADAD2025: A Dataset and Benchmark for DDoS Intrusion Detection in A2X Metaverses. In *IEEE International Conference on Computational Intelligence and Communication Networks (CICN)*, 2025.

[302] Iyas Qaddara, Insaf Kraidia, Mohammad O. Hiari, et al. A Unified Blockchain-VR Trust Architecture for Real-Time, Secure, and Fully Auditable Metaverse Environments. In *nan*, 2026.